Lernzettel zur Prüfung

KI-Coding-Agenten in Entwickler-Workflows: Gestaltungsprinzipien und Evidenzgrenzen

Matej Vidakovic · Malik Haase · Sergiu Paculea | Neue Konzepte der Informatik

Präsentation am 12.11.2026 · Abschlussveranstaltung am 13.11.2026

Prüfungsrahmen: was am 12.11. passiert

- **10 Minuten Referat** im Team, also **ca. 3 Minuten Redezeit pro Person**.
- Danach **Fachfragen**: **1–2 Fragen pro Person**, die du selbst beantworten musst.
- Gewicht: **30 %** der Note (Paper 50 %, Reviews 20 %).
- Das Format lehnt sich an eine echte Konferenz an. Erwartet werden Fragen zu **Inhalt**, **Methodik**, **Grenzen** und **Ausblick**. Fragen können an jede Person gehen, auch zu Folien, die du nicht selbst präsentierst.

So nutzt du diesen Zettel

Abschnitte 2–3: Kernaussage und eigener Redeteil. Abschnitte 4–10: Fachwissen. Abschnitt 7: alle Zahlen auf einen Blick. Abschnitt 9: jede Quelle im Detail (Forschungsfrage, Methode, Ergebnisse, Limitationen). Abschnitte 12–13: die wahrscheinlichsten Fachfragen mit Musterantworten und die typischen Denkfehler. Inhalte mit [Hintergrund, nicht im Paper] stehen nicht im Paper. Sie helfen bei Rückfragen, sollten aber vorsichtig formuliert werden.

Inhaltsverzeichnis

1 Lernplan bis zur Prüfung **4**

2 Das Paper in 30 Sekunden **4**

3 Folienablauf und Redeanteile **5**

4 Methodik: Vorgehen und Abgrenzung **7**

4.1 Was genau gemacht wurde 7

4.2 Wie ausgewertet wurde 7

4.3 Definition: Coding-Agent 8

4.4 Eigene methodische Schwächen (offen zugeben!) 8

5 Glossar: Begriffe, die du erklären können musst 9

6 Befunde nach Dimension (Herzstück) 11

6.1 Autonomie und menschliche Beteiligung / Produktivität 11

6.2 Kontext und Steuerung 12

6.3 Verifikation, Sicherheit und Integration 13

7 Alle Zahlen auf einen Blick 14

8 Die 21 Quellen: Steckbriefe 16

9 Quellen verstehen: alle Arbeiten im Detail 18

9.1 Grundlagen und Agentenbegriff 18

9.2 Produktivität und menschliche Aufsicht 21

9.3 Kontext und Steuerung 23

9.4 Benchmarks, Verifikation und Sicherheit 25

9.5 Integration in Pull Requests 29

10 Die fünf Gestaltungsprinzipien 30

11 Diskussion, Evidenzgrenzen und Ausblick 32

11.1 Was trägt die Evidenz? 32

11.2 Limitationen (Paper und Literatur) 32

11.3 Forschungslücken 33

11.4 Zentraler Zielkonflikt 33

11.5 Vorgeschlagene nächste Studie (sicher erklären können!) 33

12 Erwartete Fachfragen mit Musterantworten 34

12.1 Grundverständnis 34

12.2 Methodik und Kritik 35

12.3 Fragen zu den Befunden 36

12.4 Fragen zu den Prinzipien 39

12.5 Ausblick, Transfer und Meinung 41

12.6 Fangfragen (ruhig bleiben, präzise korrigieren) 42

13 Typische Denkfehler: niemals so sagen **43**

14 Strategie für die Fragerunde **44**

15 Schnelltest: Karteikarten **44**

16 Checkliste für den 12.11. **46**

1 Lernplan bis zur Prüfung

Tag	Aufgabe
So 08.11.	Lernzettel einmal komplett lesen. Den 30-Sekunden-Pitch (Abschnitt 2) laut üben, bis er frei sitzt.
Mo 09.11.	Befunde und Zahlen (Abschnitte 6 und 7) lernen. Zu jeder Zahl sagen können: *Was wurde gemessen? Was folgt daraus **nicht**?* Danach die Quellenkarten (Abschnitt 9) lesen, zuerst die Quellen deines eigenen Folienteils.
Di 10.11.	Prinzipien, Grenzen, Ausblick (Abschnitte 10 und 11). Danach alle Fachfragen (Abschnitt 12) beantworten, ohne die Musterantwort anzuschauen, und erst anschließend vergleichen.
Mi 11.11.	**Generalprobe mit dem Team** und Stoppuhr (Ziel: 9:30–10:00 min). Gegenseitig Fachfragen stellen. Übergaben zwischen den Sprechern üben. Klären, wer bei welchem Fragentyp ergänzen darf.
Do 12.11.	Morgens nur noch: Pitch, Zahlen-Tabelle, Fallen-Liste (Abschnitt 13). Nichts Neues mehr lernen.

2 Das Paper in 30 Sekunden

Auswendig können: der Elevator Pitch

„Wir haben in einer narrativen Literaturanalyse **21 Arbeiten von 2024 bis 2026** zu KI-Coding-Agenten ausgewertet. Entlang der drei Dimensionen **Autonomie, Kontext und Qualitätssicherung** leiten wir **fünf Gestaltungsprinzipien** ab: gemeinsame Planung, risikogerechte Autonomie, gezielter Kontext, mehrstufige Verifikation und überprüfbare Integration. Gleichzeitig zeigen wir, dass die Literatur nur **einzelne, eng begrenzte Maßnahmen** kontrolliert belegt. Ein **allgemeiner Produktivitäts- oder Qualitätsgewinn** lässt sich nicht ableiten. Deshalb müssen aktive Arbeitszeit, Gesamtdauer, Testbestehen und funktionale Korrektheit **getrennt** gemessen werden.“

Forschungsfrage (wörtlich)

Welche Gestaltungsprinzipien für Entwickler-Workflows mit KI-Coding-Agenten lassen sich aus aktueller Literatur ableiten, und welche empirischen Hinweise und Evidenzgrenzen bestehen hinsichtlich Codequalität und Produktivität?

Antwort auf die Forschungsfrage: zweigeteilt

1. **Teil 1 (Prinzipien):** Ja, konkrete Workflow-Empfehlungen sind ableitbar, nämlich die fünf Prinzipien P1–P5.
2. **Teil 2 (Evidenz):** Kontrollierte Belege gibt es nur für einzelne Maßnahmen (z. B. Linter in SWE-agent). Die Wirksamkeit der Prinzipien **als Gesamtworkflow** und ein **allgemeiner Gewinn** an Qualität oder Produktivität sind **nicht nachgewiesen**.

Die drei Kernbotschaften

1. **Weniger aktive Arbeit** $\neq$ **kürzere Gesamtdauer** (Chen et al.)
2. **Bestandene Tests** $\neq$ **korrekte Lösung** (AutoCodeRover, Wang et al.)
3. **Mehr Kontext** $\neq$ **mehr Erfolg**, aber höhere Kosten (Gloaguen et al.)

Beitrag des Papers

Das Paper liefert eine **vergleichende Synthese**, keine eigene empirische Studie. Das Neue daran:

- Es **trennt Evidenzarten** (kontrollierte Vergleiche, deskriptive Beobachtungen, qualitative Aussagen).
- Es **trennt Endpunkte** (aktive Zeit, Gesamtdauer, Kosten, Testbestehen, Korrektheit, Sicherheit).
- Es kennzeichnet die Handlungsregeln **ausdrücklich als eigene Vorschläge** und nicht als belegte Wirkungsaussagen.

3 Folienablauf und Redeanteile

Vorschlag zur Aufteilung (mit dem Team abstimmen)

Person A: Folien 1–4 (Einstieg, Motivation, Forschungsfrage, Vorgehen), ca. 3 min.
Person B: Folien 5–7 (die drei Befunde), ca. 3,5 min.
Person C: Folien 8–11 (Prinzipien, Evidenzgrenzen, Zielkonflikt, Fazit), ca. 3,5 min.
Folie 12 (Literatur) bleibt während der Fragerunde stehen. **Lerne trotzdem alle Folien**, denn die Fragen können an jede Person gehen.

Nr.	Folie	Kernsatz, den du sagen können musst
1	Titel	21 aktuelle Arbeiten ausgewertet, fünf Prinzipien abgeleitet. Ebenso wichtig ist die Frage, was davon belegt ist und was nicht.
2	Motivation	22,20 % von 128.018 GitHub-Projekten nutzen Coding-Agenten (konservativ, obere Schätzung 28,66 %). Arbeit verlagert sich zum *Spezifizieren, Überwachen, Prüfen*. Benchmarks und PR-Zahlen beantworten nicht, ob das Zeit spart.
3	Forschungsfrage	Zweigeteilte Frage (Prinzipien und Evidenzgrenzen). Drei Dimensionen: Autonomie, Kontext, Qualitätssicherung. Die Prinzipien sind begründete Empfehlungen und nicht als Workflow nachgewiesen.
4	Vorgehen	ACM DL, IEEE Xplore, arXiv plus Rückwärtssuche. Ab 2024, mehrschrittige werkzeugnutzende LLM-Agenten. Evidenzarten getrennt, keine studienübergreifenden Effektgrößen. Narrativ, keine systematische Vollerhebung.
5	Befund 1: Produktivität	Chen: Erfolg 25 % $\rightarrow$ 60 %, aktive Zeit 25,1 $\rightarrow$ 12,5 min, aber mit Agentenlaufzeit 27,9 min. Becker: +19 % Bearbeitungszeit, obwohl die Entwickler 20 % Ersparnis schätzten.
6	Befund 2: Kontext	Gloaguen: kein signifikanter Gesamteffekt gegenüber keiner Kontextdatei. LLM-generierte Dateien: Kosten +20/+23 %, Schritte +2,45/+3,92. Nur Anforderungen ergänzen, die der Agent sonst nicht findet.
7	Befund 3: Verifikation	51 von 78 (65,4 %) testbestehenden AutoCodeRover-Patches korrekt. 29,6 % von 877 Patches weichen im Verhalten ab. 214 von 475 (45,1 %) gemergten Claude-Code-PRs wurden vorher revidiert.
8	Prinzipien	P1–P5, jeweils mit Handlungsregel (eigener Vorschlag) und Begründungsquelle.
9	Evidenzgrenzen	Kontrolliert belegt: Linter 15 $\rightarrow$ 18 %, Agent vs. Copilot 25 $\rightarrow$ 60 %, ChatDev-Topologie 25,0 $\rightarrow$ 40,6 % (mehrere Änderungen zugleich). Offen: Gesamtworkflow, Nicht-Python, Langzeit-Wartbarkeit und Prüfaufwand.
10	Zielkonflikt	Zusätzliche Prüfung lohnt sich nur, wenn sie genug Defekte findet. Nächster Schritt: kontrollierter Vergleich mit/ohne Planfreigabe bei gleichem Agenten, Modell und Budget. Getrennt messen: Defekte, aktive Zeit, Gesamtdauer, Review-Aufwand.

Nr.	Folie	Kernsatz, den du sagen können musst
11	Fazit	Konkrete Empfehlungen sind möglich, ein allgemeiner Produktivitätsgewinn ist nicht belegt. Nötig sind kontrollierte Einzelvergleiche und Langzeitmessungen.

4 Methodik: Vorgehen und Abgrenzung

4.1 Was genau gemacht wurde

- **Art:** gezielte **narrative Literaturanalyse** (kein systematisches Review, keine Meta-Analyse).
- **Korpus: 21 Arbeiten**, Erscheinungsjahre **2024–2026**, Literaturstand **16. Oktober 2026**.
- **Datenbanken:** ACM Digital Library, IEEE Xplore, arXiv, dazu **Rückwärtssuche** (Backward Snowballing) in den Literaturverzeichnissen.
- **Suchbegriffe:** *coding agent, software engineering agent, agentic coding, LLM-based agent*, kombiniert mit *workflow, human-in-the-loop, autonomy, code review, testing, productivity, code quality*.
- **Ablauf:** Duplikate entfernen, dann Titel/Abstract prüfen, dann Volltext prüfen.
- **Einschlusskriterien:** englisch oder deutsch, ab 2024. Untersucht werden **mehrschrittig arbeitende, werkzeugnutzende LLM-Agenten**, entweder empirisch oder als System mit Evaluation. Es muss ein Bezug zu Workflow, Qualität oder Produktivität bestehen.
- **Mehrere Fassungen:** Zitiert wird die begutachtete Version. Wurde inhaltlich ein Preprint ausgewertet, ist dessen Version angegeben.
- **Quellentypen:** Konferenzbeiträge sind die technische Grundlage. Journals und gekennzeichnete Preprints ergänzen Praxis- und Nutzerbefunde.
- **Codevervollständigung** (Copilot-Autocomplete) dient **nur als angrenzender Produktivitätsvergleich**.

4.2 Wie ausgewertet wurde

Für jede Studie wurden **Studiendesign, Werkzeug, Population bzw. Benchmark, Endpunkt und Einschränkungen** gegenübergestellt. Drei Evidenzarten werden getrennt eingeordnet:

1. **Kontrollierte Vergleiche** (z. B. Nutzerexperiment, Ablation, RCT): stärkste Aussagekraft zu Ursache und Wirkung.
2. **Deskriptive Beobachtungen** (z. B. Repository- und PR-Analysen): praxisnah, aber nicht kausal.
3. **Qualitative Aussagen** (z. B. 17 Interviews): erklären Praktiken, erlauben aber keine Häufigkeitsschätzung.

Warum keine studienübergreifenden Effektgrößen?

Die Studien messen **verschiedene Endpunkte** (aktive Zeit, Gesamtzeit, PRs/Woche, Erfolgsrate, Kosten) mit **verschiedenen Werkzeugen, Modellen, Budgets und Populationen**. Beispiel aus dem Paper: Ein Architekturvergleich isoliert keinen Autonomieeffekt, wenn gleichzeitig Modell, Werkzeuge oder Budget variieren. Deshalb gibt es **keine Mittelung und kein Ranking**.

4.3 Definition: Coding-Agent

Ein **auf einem großen Sprachmodell (LLM) beruhendes System**, das **Entwicklungsaktionen mit Werkzeugen ausführt** (Dateien lesen/ändern, Befehle, Tests) und **Rückmeldungen der Umgebung für weitere Schritte nutzt** (Liu et al. 2024, Survey).

Spektrum des Autonomiegrads:

- **Agentless** (Xia et al. 2025): feste Phasen *Lokalisierung, Reparatur, Validierung*. Der Ablauf ist vorgegeben, das LLM entscheidet nicht frei. Wenig Autonomie.
- **HULA** (Takerngsaksiri et al. 2025): Human-in-the-loop. Menschliche Rückmeldungen fließen in Planung und Umsetzung ein, im industriellen Einsatz (Atlassian/Jira [Hintergrund, nicht im Paper]).
- **OpenHands** (Wang et al. 2025): offene Plattform mit isolierten Ausführungsumgebungen (Docker-Sandbox). Hohe Autonomie.

4.4 Eigene methodische Schwächen (offen zugeben!)

- **Trefferzahlen werden nicht berichtet**, daher **keine reproduzierbare systematische Vollerhebung** (kein PRISMA-Flussdiagramm).
- Die **narrative Auswahl** begrenzt Vollständigkeit und Reproduzierbarkeit.
- **Preprints** liefern wesentliche Nutzer- und Sicherheitsbefunde (Becker, Dhanorkar, Sajadi, Li, Aleithan, Liu). Ihr Status muss bei der Gewichtung beachtet werden.
- Modelle, Agentenversionen und Budgets unterscheiden sich zwischen den Studien.
- **SWE-bench-Dominanz** (Python-Repositories) begrenzt die Übertragbarkeit auf andere Sprachen und Industrieaufgaben.

5 Glossar: Begriffe, die du erklären können musst

Endpunkt
Die Zielgröße, die eine Studie misst (z. B. Erfolgsrate, aktive Zeit, Gesamtdauer, Kosten, Sicherheitslücken). **Zentrale These des Papers:** Endpunkte dürfen nicht vermischt werden.

Aktive Arbeitszeit vs. Gesamtdauer
Aktive Zeit ist die Zeit, in der der Mensch arbeitet. Die Gesamtdauer enthält zusätzlich die Laufzeit des Agenten (Warten). Chen: 12,5 min aktiv, aber 27,9 min gesamt.

Funktionale Korrektheit vs. Testbestehen
Testbestehen ist nur ein **Näherungsmaß** (Proxy). Tests können schwach sein oder Anforderungen nicht vollständig abdecken.

SWE-bench
Benchmark mit **realen GitHub-Issues**. Ein Agent muss einen Patch erzeugen, der anhand von Tests geprüft wird (Jimenez et al. 2024). [Hintergrund, nicht im Paper]: ca. 2.294 Aufgaben aus 12 Python-Repositories.

SWE-bench Lite / Verified
Teilmengen. [Hintergrund, nicht im Paper]: *Lite* umfasst 300 Aufgaben (günstiger zu evaluieren). *Verified* umfasst 500 von Menschen geprüfte, lösbare Aufgaben.

CTXBench
Benchmark von Gloaguen et al. speziell für Kontextdateien.

SWE-Bench+
Analyse von Aleithan et al. Benchmark-Risiken: **Lösungen stehen schon in Issue-Beschreibung/Kommentaren** (Solution Leakage) und **schwache Tests**.

@1 / @3
Erfolg im **einzelnen Lauf** (@1) bzw. **aggregiert über drei Versuche** (@3, Erfolg, wenn mindestens ein Versuch klappt). Die beiden Werte dürfen nicht vermischt werden.

Differenzielles Testen
Das Verhalten des Agenten-Patches wird mit dem der Referenzlösung verglichen (gleiche Eingaben, verschiedene Ausgaben?). **Eine Abweichung ist kein Fehlerbeweis**, denn auch alternative korrekte Lösungen weichen ab.

Ablation
Eine einzelne Komponente wird entfernt/hinzugefügt, alles andere bleibt gleich. So lässt sich der **Effekt isolieren**. Beispiel: SWE-agent mit vs. ohne Linter.

Linter
Statisches Prüfwerkzeug (Syntax, Stil, einfache Fehler), ohne den Code auszuführen. In SWE-agent werden fehlerhafte Edits abgewiesen.

Kontextdatei
Datei im Repository mit Anweisungen für den Agenten: `CLAUDE.md`, `AGENTS.md`. Inhalt: Konventionen, Architekturhinweise, Build-/Testbefehle.

Sandbox
Isolierte Laufzeitumgebung (z. B. Docker bei OpenHands). Sie schützt das **System** vor Agentenaktionen, macht aber den **erzeugten Code nicht sicher**.

Human-in-the-loop
Der Mensch ist an definierten Punkten in den Agentenablauf eingebunden (Plan, Umsetzung, Review).

Aufsichtsformen
Laut Dhanorkar: **Vorgaben vorab, gemeinsame Planung, laufende Überwachung, nachträgliches Review**.

RCT / randomisiertes Experiment
Die Zuteilung zur Bedingung (z. B. KI erlaubt/verboten) erfolgt zufällig. Das ist der Goldstandard für Kausalaussagen. Beispiele: Becker, Cui.

Feldexperiment
Ein randomisiertes Experiment in realer Arbeitsumgebung statt im Labor (Cui: in Unternehmen).

Statistisch nicht signifikant (n. s.)
Der beobachtete Unterschied ist mit Zufall vereinbar. **Das ist kein Nachweis von Gleichheit** („absence of evidence $\neq$ evidence of absence“).

Standardfehler
Maß für die Unsicherheit einer Schätzung. Cui: 26,08 % $\pm$ 10,3 % SE, das grobe 95 %-Intervall liegt bei ca. 6 % bis 46 % (selbst gerechnet: $26{,}08 \pm 1{,}96 \cdot 10{,}3$).

Prozentpunkte (PP) vs. Prozent
25 % $\rightarrow$ 60 % sind **+35 PP**, relativ aber **+140 %**. Immer sauber trennen!

Preprint
Nicht (oder noch nicht) begutachtete Veröffentlichung, z. B. auf arXiv.

Narratives vs. systematisches Review
Ein narratives Review wählt gezielt und interpretierend aus. Ein systematisches Review folgt einem vollständig dokumentierten, reproduzierbaren Protokoll (z. B. PRISMA, Trefferzahlen pro Schritt).

Rückwärtssuche (Backward Snowballing)
Weitere Quellen werden über die Literaturverzeichnisse bereits gefundener Arbeiten gesucht.

Schließlatenz
Zeit vom Öffnen bis zum Schließen eines PRs. Sie sagt **nichts über menschliche Arbeitszeit oder Durchsatz**.

Revision vor Merge
Ein PR wird vor der Übernahme noch geändert, also Nacharbeit (Watanabe: 45,1 %, häufigste Kategorie Bugfixes).

Multi-Agent-Topologie
Wie mehrere LLM-Agenten zusammenarbeiten (Rollen, Reihenfolge, Iterationen). ChatDev simuliert eine „Softwarefirma“ aus Rollen-Agenten [Hintergrund, nicht im Paper].

Codevervollständigung vs. Agent
Autocomplete schlägt Zeilen/Blöcke im Editor vor, der Mensch steuert alles. Ein Agent plant und handelt **mehrschrittig** mit Werkzeugen und nutzt Feedback der Umgebung.

Data Contamination
[Hintergrund, nicht im Paper]: Benchmark-Aufgaben/Lösungen waren evtl. in den Trainingsdaten des Modells (öffentliche GitHub-Repos). Ergebnisse können dadurch überschätzt werden.

6 Befunde nach Dimension (Herzstück)

6.1 Autonomie und menschliche Beteiligung / Produktivität

Chen et al. (CHI ’26): OpenHands vs. Copilot

- **Design:** kontrollierte Nutzerstudie, **20 Studierende**.
- **Erfolg:** korrekt abgeschlossene Aufgaben **25 % (Copilot)** → **60 % (OpenHands)**, also +35 PP.
- **Aktive menschliche Zeit** (nur erfolgreiche Aufgaben): **25,1 min (Copilot)** → **12,5 min (OpenHands)**.
- **Inklusive Agentenaktionen:** OpenHands **27,9 min**, also *länger* als die 25,1 min aktive Zeit mit Copilot.
- **Grenze:** unterschiedliche Erfolgsquoten (Zeitvergleich nur über erfolgreiche Aufgaben, daher verzerrt), kleine Studierendenstichprobe. Nur begrenzt auf professionelle Abläufe übertragbar.

Cui et al. (Management Science 2026): Copilot-Codevervollständigung

- **Design: drei randomisierte Feldexperimente**, **4.867 Entwickler** ([Hintergrund, nicht im Paper]: u. a. Microsoft, Accenture, ein Fortune-100-Unternehmen).

- **Befund:** gepoolt **+26,08 % erledigte Aufgaben** (gemessen als wöchentliche PRs), Standardfehler 10,3 %.
- **Grenze: kein Test autonomer Agenten** und kein Test der Prinzipien. Nur angrenzender Vergleich.

Becker et al. (METR, Preprint 2025)

- **Design:** randomisierte KI-Zulassung pro Aufgabe, **16 erfahrene Open-Source-Entwickler**, **246 Aufgaben**, überwiegend **Cursor**, Werkzeuge von **Anfang 2025**.
- **Befund:** Bearbeitungszeit mit erlaubter KI **+19 % (langsamer)**, nachträglich aber **20 % Zeitersparnis geschätzt**.
- **Grenze:** vertraute eigene Projekte, Werkzeugstand Anfang 2025, kleine Population.
- [Hintergrund, nicht im Paper]: Vorab erwarteten die Teilnehmenden ca. 24 % Beschleunigung.

Cui vs. Becker ist kein Widerspruch

Sie unterscheiden sich in **Werkzeugtyp** (Autocomplete vs. Cursor/agentische Werkzeuge), **Population** (breite Belegschaft vs. 16 Experten in eigenen Projekten) und **Zeitmaß** (PRs pro Woche vs. Bearbeitungszeit). Folgerung: Diese drei Faktoren müssen bei der Übertragung auf agentische Workflows berücksichtigt werden.

Menschliche Aufsicht

- **HULA** (ICSE-SEIP 2025): Rückmeldungen zu Plänen und Implementierungen im industriellen Einsatz.
- **Dhanorkar et al.** (Preprint 2026, 17 Interviews): vier Aufsichtsformen (Vorgaben vorab, gemeinsame Planung, laufende Überwachung, nachträgliches Review). Entwickler nutzen das **Vertrauen auf Pläne als Prüfheuristik**. Die Studie misst **weder Fehlerrate noch Qualitätseffekt**.
- **Folgerung: Freigegebene Pläne belegen keine korrekte Umsetzung.**

6.2 Kontext und Steuerung

- **Chatlatanagulchai et al.** (PROFES 2025): Analyse von **253** `CLAUDE.md`**-Dateien**. Inhalte: Projektkonventionen, Architekturhinweise, Build-Befehle.
- **Mohsenimofidi et al.** (MSR 2026): Kontextdateien in **466 von 10.000** Open-Source-Projekten (ca. 4,7 %). Misst **nur Verbreitung**, keine Leistungseffekte.

- **Gloaguen et al.** (ICLR 2026 Workshop): Kontextvarianten auf **SWE-bench Lite** und **CTXBench**.
  - Gegenüber **fehlender** Kontextdatei: **kein statistisch signifikanter Gesamteffekt** auf die Erfolgsrate (**kein Gleichheitsnachweis!**).
  - CTXBench: **entwicklerseitig eingecheckte** Dateien sind **signifikant besser als LLM-generierte**, aber **nicht signifikant besser als gar keine**.
  - LLM-generierte Dateien: **Schritte +2,45** (Lite) / **+3,92** (CTXBench), **Inferenzkosten +20 %** (Lite) / **+23 %** (CTXBench).
  - Empfehlung der Autoren: zusätzliche Anweisungen auf **konkrete, sonst nicht verfügbare Anforderungen** begrenzen.
- **Folgerung:** Schritte, Kosten und Erfolg sind getrennte Endpunkte. Ein **generell optimaler Kontextumfang ist nicht bestimmt**.

6.3 Verifikation, Sicherheit und Integration

Testbestehen ist nur ein Näherungsmaß

- **SWE-bench** prüft Reparaturen realer Issues anhand von Tests (Jimenez et al. 2024).
- **AutoCodeRover** (Zhang et al., ISSTA 2024): Im aggregierten **@3**-Versuch wurden **51 von 78** testbestehenden Patches manuell als korrekt eingestuft (**65,4 %**). Umgekehrt waren 27 Patches (34,6 %) trotz bestandener Tests nicht als korrekt bewertet. **Nicht mit @1-Erfolgsrate vermischen!**
- **SWE-Bench+** (Aleithan et al., Preprint): Lösungen stehen in Issues/Kommentaren, Tests sind schwach. Beides sind Benchmark-Risiken.
- **Wang, Pradel, Liu** (ICSE 2026): **877** testbestehende Patches auf SWE-bench Verified.
  - **7,8 %** scheitern an der **vollständigen** Entwicklertestsuite (ca. 68 Patches).
  - Differenzielle Tests: **260 Patches (29,6 %)** zeigen Verhaltensabweichungen zur Referenz.
  - **Abweichung $\neq$ Fehler:** Von **77** manuell untersuchten verdächtigen Patches sind **22 inkorrekt, 4 korrekt, 51 unklar**.

Prüfmechanismen mit konkreten Effekten

- **SWE-agent** (Yang et al., NeurIPS 2024): Ablation **mit Linter 18 % statt 15 %** gelöste Aufgaben auf SWE-bench Lite (+3 PP, relativ +20 %). Das ist der **stärkste direkte, isolierte Hinweis** im Korpus.
- **Cemri et al.** (NeurIPS 2025 D&B): **32 Aufgaben** aus ProgramDev-v0, veränderte iterative **ChatDev-Topologie**: **25,0 %** $\rightarrow$ **40,6 %** (+15,6 PP). Das ist der **Effekt mehrerer**

Änderungen gleichzeitig und **kein isolierter Nachweis** für eine abschließende Zielprüfung. [Hintergrund, nicht im Paper]: Die Arbeit stellt die Fehlertaxonomie MAST für Multi-Agenten-Systeme vor.

Sicherheit (eigener Endpunkt)

- **Sajadi et al.** (Preprint 2025): Sicherheitslücken in Reparaturen mehrerer Agenten.
- Unterschiedliche Modelle und Bearbeitungsmechanismen, daher **keine kausale Zuschreibung an den Autonomiegrad**.
- Bei **Llama-Patches**: mehr geänderte Dateien gehen mit mehr Sicherheitsproblemen einher (**Assoziation**). Eine **Schutzwirkung kleinerer Patches wurde nicht gemessen**.

Integration (Pull Requests)

- **Watanabe et al.** (TOSEM 2026): **475 von 567** Claude-Code-PRs gemergt (ca. 84 %). Davon **214 vor dem Merge revidiert (45,1 %)**. Häufigste Revisionskategorie: **Bugfixes 47,7 %** (Mehrfachzuordnung möglich). Der Unterschied im Revisionsumfang zu menschlichen PRs ist **nicht signifikant**, was **keine Gleichheit** beweist.
- **Li et al.** (AIDev, Preprint 2025): **kürzere Schließlatenzen** für angenommene Codex-PRs. Das misst **weder menschliche Arbeitszeit noch Durchsatz**. Eine explizite GitHub-Reviewaktion fehlt bei **58,2 %** der Agenten-PRs und bei **75,3 %** der menschlichen Vergleichs-PRs. Private Prüfung oder Prüftiefe sind daraus **nicht ablesbar**.

7 Alle Zahlen auf einen Blick

Zahl	Bedeutung	Quelle / Was daraus *nicht* folgt
21	ausgewertete Arbeiten, 2024–2026	Narrativ, keine Vollerhebung
16.10.2026	Literaturstand	–
128.018	untersuchte GitHub-Projekte	Robbes et al.
22,20 %	konservative Adoptionsrate von Coding-Agenten	nicht repräsentativ für alle Softwareentwicklung
28,66 %	hochgerechnete obere Schätzung	Robbes et al.
20	Studierende in der Nutzerstudie	Chen et al.: klein, nicht professionell

Zahl	Bedeutung	Quelle / Was daraus *nicht* folgt
25 → 60 %	korrekte Abschlüsse Copilot → OpenHands (+35 PP)	Chen et al.
25,1 → 12,5 min	aktive menschliche Zeit (erfolgreiche Aufgaben)	aktive Zeit ist keine Gesamtdauer
27,9 min	OpenHands inkl. Agentenaktionen	länger als 25,1 min!
4.867	Entwickler in 3 randomisierten Feldexperimenten	Cui et al.: Autocomplete, kein Agent
+26,08 %	mehr erledigte Aufgaben (wöchentl. PRs), SE 10,3 %	keine Aussage über Agenten
16 / 246	erfahrene OSS-Entwickler / Aufgaben	Becker et al. (METR)
+19 %	längere Bearbeitungszeit mit KI	Becker: Tools Anfang 2025, vertraute Projekte
20 %	subjektiv geschätzte Ersparnis (nachträglich)	Selbsteinschätzung unzuverlässig
17	Interviews zu Aufsicht	Dhanorkar: qualitativ, keine Häufigkeiten
253	analysierte `CLAUDE.md`-Dateien	Chatlatanagulchai et al.
466 / 10.000	Projekte mit Kontextdateien	Mohsenimofidi: nur Verbreitung
n. s.	Kontextdatei vs. keine (Erfolgsrate)	Gloaguen: kein Gleichheitsnachweis
+2,45 / +3,92	mehr Schritte durch LLM-generierte Kontextdateien (Lite / CTXBench)	Gloaguen et al.
+20 / +23 %	mehr Inferenzkosten (Lite / CTXBench)	Gloaguen et al.
51 / 78 = 65,4 %	testbestehende AutoCodeRover-Patches manuell korrekt (@3)	nicht mit @1 vermischen
877	testbestehende Patches auf SWE-bench Verified	Wang, Pradel, Liu
7,8 %	scheitern an vollständiger Testsuite	Wang et al.
260 = 29,6 %	Verhaltensabweichung (differenzielles Testen)	Abweichung $\neq$ Fehler
77: 22 / 4 / 51	manuell geprüft: inkorrekt / korrekt / unklar	Wang et al.
15 → 18 %	SWE-agent ohne → mit Linter (SWE-bench Lite)	isolierte Ablation

Zahl	Bedeutung	Quelle / Was daraus *nicht* folgt
32	Aufgaben aus ProgramDev-v0	Cemri et al.
25,0 → 40,6 %	veränderte ChatDev-Topologie (+15,6 PP)	mehrere Änderungen, nicht isoliert
567 / 475	Claude-Code-PRs gesamt / gemergt	Watanabe et al.
214 = 45,1 %	gemergte PRs vor Merge revidiert	Merge ≠ fehlerfrei
47,7 %	Bugfix-Anteil an Revisionen (Mehrfachzuordnung)	Watanabe et al.
58,2 / 75,3 %	ohne explizite GitHub-Reviewaktion (Agent / Mensch)	Li et al.: private Prüfung unsichtbar

Eselsbrücken

- **„25–60, 25–12–28“**: Chen (Erfolg 25 → 60, Zeit 25 → 12,5, gesamt 27,9).
- **„+19 gemessen, −20 gefühlt“**: Becker.
- **„20/23 Kosten, 2,45/3,92 Schritte“**: Gloaguen, immer Reihenfolge Lite / CTX-Bench.
- **„Zwei Drittel korrekt“**: AutoCodeRover 65,4 %.
- **„Fast jeder Dritte weicht ab“**: Wang 29,6 %.
- **„Fast jeder Zweite revidiert“**: Watanabe 45,1 %.
- **„15 auf 18“**: Linter, der sauberste Einzelbefund.

8 Die 21 Quellen: Steckbriefe

Konferenz-Kürzel: ICSE, FSE, ISSTA, MSR = Software-Engineering-Konferenzen. NeurIPS, ICLR = Machine-Learning-Konferenzen. CHI = Human-Computer-Interaction. TOSEM = ACM-Journal. Reine Preprints im Korpus: Liu, Becker, Dhanorkar, Aleithan, Sajadi, Li.

Quelle	Venue / Typ	Kernaussage und Rolle im Paper
Robbes et al. 2026	TOSEM, Repository-Analyse	Adoption 22,20 % (bis 28,66 %) von 128.018 GitHub-Projekten. *Relevanz/Motivation.*
Liu et al. 2024	arXiv-Survey	Definition LLM-basierter SE-Agenten. *Begriffsdefinition.*

Quelle	Venue / Typ	Kernaussage und Rolle im Paper
Xia et al. 2025 (Agentless)	FSE 2025, System + Eval.	Feste Phasen Lokalisierung, Reparatur, Validierung. *Pol geringer Autonomie.*
Takerngsaksiri et al. 2025 (HULA)	ICSE-SEIP 2025, Industrie	Menschliches Feedback zu Plan und Umsetzung. *Stützt P1 (Umsetzbarkeit).*
Wang et al. 2025 (OpenHands)	ICLR 2025, Plattform	Offene Agentenplattform mit Docker-Sandbox. *P2: Sandbox $\neq$ sicherer Code.*
Chen et al. 2026	CHI '26, kontr. Nutzerstudie	OpenHands vs. Copilot: 25 $\rightarrow$ 60 %, aktive Zeit 25,1 $\rightarrow$ 12,5 min, gesamt 27,9 min. *Befund 1.*
Cui et al. 2026	Management Science, 3 RCT-Feldexp.	+26,08 % Aufgaben mit Autocomplete. *Angrenzender Vergleich.*
Becker et al. 2025 (METR)	arXiv, RCT	+19 % Zeit, 20 % gefühlte Ersparnis. *Gegenbefund zu Cui.*
Dhanorkar et al. 2026	arXiv, 17 Interviews	Vier Aufsichtsformen, Plan-Vertrauen als Heuristik. *Arbeitsverlagerung, P1, P2.*
Chatlatanagulchai et al. 2025	PROFES 2025, Dateianalyse	253 `CLAUDE.md`: Konventionen, Architektur, Build. *Was in Kontextdateien steht.*
Mohsenimofidi et al. 2026	MSR 2026, Repo-Analyse	466/10.000 Projekte mit Kontextdateien. *Verbreitung, P3.*
Gloaguen et al. 2026	ICLR-Workshop, kontr. Experiment	n. s. Erfolg, +20/23 % Kosten, +2,45/3,92 Schritte. *Befund 2, P3.*
Jimenez et al. 2024 (SWE-bench)	ICLR 2024, Benchmark	Reale GitHub-Issues, Prüfung durch Tests. *Messinstrument, Python-Grenze.*
Zhang et al. 2024 (AutoCodeRover)	ISSTA 2024, System + Eval.	51/78 = 65,4 % testbestehend und korrekt (@3). *Befund 3.*
Aleithan et al. 2024 (SWE-Bench+)	arXiv, Benchmark-Analyse	Solution Leakage, schwache Tests. *Benchmark-Kritik.*
Wang, Pradel, Liu 2026	ICSE 2026, emp. Patch-Analyse	877 Patches: 7,8 % volle Suite, 29,6 % abweichend, 22/4/51. *Befund 3, P4.*
Yang et al. 2024 (SWE-agent)	NeurIPS 2024, System + Ablation	Linter 15 $\rightarrow$ 18 %. *Stärkster Einzelbefund, P4.*
Cemri et al. 2025 (MAST)	NeurIPS 2025 D&B, Fehleranalyse	ChatDev 25,0 $\rightarrow$ 40,6 %, mehrere Änderungen. *P4: Gesamtauftrag prüfen.*

Quelle	Venue / Typ	Kernaussage und Rolle im Paper
Sajadi et al. 2025	arXiv, Sicherheitsanalyse	Sicherheitslücken in Agenten-Patches, Llama: mehr Dateien, mehr Probleme (Assoziation). *P2, P5.*
Watanabe et al. 2026	TOSEM, PR-Analyse	475/567 gemergt, 45,1 % revidiert, Bugfixes 47,7 %. *Befund 3, P5.*
Li et al. 2025 (AIDev)	arXiv, PR-Datensatz	Kürzere Schließlatenz für Codex-PRs, 58,2/75,3 % ohne Reviewaktion. *Proxy-Kritik.*

9 Quellen verstehen: alle Arbeiten im Detail

Wozu dieser Abschnitt?

Fragen wie „Was hat Gloaguen eigentlich gemacht?“ oder „Wie wurde bei Becker gemessen?“ kannst du nur beantworten, wenn du die Originalarbeiten verstehst. Jede Karte enthält: **Worum geht's** (Abstract-Kern), **Forschungsfrage/Ziel**, **Methode**, **Daten**, **Ergebnisse**, **Limitationen**, **Begriffe** und die **Rolle in unserem Paper**.
Quelle der Karten: Auswertung der Original-PDFs im Notebook „SEMINAR CLAUDE“. Zahlen, die *nicht* in unserem Paper stehen, sind Zusatzwissen. Nenne sie in der Prüfung nur, wenn du sie sicher weißt.

Venue-Angaben: Paper vs. PDF

Mehrere PDFs im Notebook sind **Preprint-Fassungen** (z. B. Agentless, HULA, Chen, Cui, Chatlatanagulchai). Unser Paper zitiert, wie in der Methodik beschrieben, die **begutachtete Veröffentlichung** (FSE 2025, ICSE-SEIP 2025, CHI '26, Management Science, PROFES 2025). Wird danach gefragt: „Ausgewertet wurde teils die Preprint-Fassung, zitiert ist die begutachtete Version.“

9.1 Grundlagen und Agentenbegriff

Robbes et al.: „Agentic Much?“ TOSEM 2026 · Repository-Mining

Worum geht's: Erste großflächige empirische Studie zur Verbreitung autonomer Coding-Agenten auf GitHub. Die Adoption ist breit über Sprachen, Projektgrößen und Organisationen gestreut.

Forschungsfrage/Ziel: Wie hoch ist die Adoptionsrate von Coding-Agenten? In welchen Kontexten (Sprachen, Themen, Organisationen) werden sie genutzt, wie entwickelt sich das zeitlich, und wie sehen Agenten-Beiträge aus?

Methode: Repository-Mining über GitHub-APIs und Git-Logs. Agenten-Spuren werden über **196 Heuristiken** in 4 Kategorien erkannt (Guidance-Dateien wie `CLAUDE.md`, Commit-Autorenschaft, Branch-Namen, PR-Labels) für 63 Agenten.

Daten: **128.018** aktive Repositories (mind. 10 Sterne, 5.000 LOC, 100 Commits, aktiv Jan 2025 – Feb 2026). Zusätzlich 790 Claude-Code-Commits manuell analysiert.

Ergebnisse: Adoption **22,20 %** (konservativ) bis **28,66 %** (Stand Feb. 2026). Führend: Copilot (13.890 Projekte), Claude Code (12.053). KI-Commits sind größer (Median 31 vs. 11 hinzugefügte Zeilen) und betreffen vor allem Features (35,7 %) und Bugfixes (29,9 %).

Limitationen: *Partial Observability*: Wer Spuren entfernt oder manuell committet, wird nicht erfasst (Untererfassung). Open-Source-Fokus bildet Industrie nur bedingt ab. Langzeitfolgen für Codequalität offen.

Begriffe: *Guidance File* (Kontextdatei), *Code Churn* (hinzugefügter und gelöschter Code).

Rolle in unserem Paper: Motivation (Folie 2): Das Thema ist in der Praxis angekommen. Grenze: nicht repräsentativ für die gesamte Softwareentwicklung.

Liu et al.: Survey zu LLM-basierten SE-Agenten

arXiv 2024 · Literatur-Survey

Worum geht's: Systematischer Überblick über LLM-basierte Agenten in der Softwaretechnik, geordnet nach SE-Aufgaben und nach Agentenarchitektur (Planung, Gedächtnis, Wahrnehmung, Aktion, Multi-Agent).

Forschungsfrage/Ziel: Wie lassen sich LLM-Agenten über den Software-Lebenszyklus klassifizieren, und was können sie gegenüber reinen LLMs zusätzlich?

Methode: Systematische DBLP-Keyword-Suche (57 Suchläufe, 10.362 Treffer), Vorwärts- und Rückwärts-Snowballing, Ein-/Ausschlusskriterien, Validierung durch Autoren-Feedback.

Daten: **124 Arbeiten** (ca. 75 % begutachtet, 25 % Preprints).

Ergebnisse: 59,7 % der Arbeiten nutzen Multi-Agenten-Systeme. Feedback in natürlicher Sprache bringt 2–17 % Leistungsplus pro Runde.

Limitationen: Subjektivität bei der Auswahl, viele Preprints, Mangel an realistischen Benchmarks und an Evaluationen einzelner Module.

Begriffe: *LLM-basierter Agent*: LLM als zentrale Steuerung plus Werkzeuge, Planung, Gedächtnis und Interaktion mit der Umgebung.

Rolle in unserem Paper: Liefert unsere **Definition** von Coding-Agent. Kontrast: Der Survey ist systematisch (mit Trefferzahlen), unser Paper narrativ.

Xia et al.: Agentless

FSE 2025 (PDF: arXiv 2024) · System + Evaluation

Worum geht's: Stellt infrage, ob komplexe autonome Agenten nötig sind. Agentless nutzt einen festen Dreischritt ohne autonome Entscheidungen des LLMs und ist dabei

günstig und konkurrenzfähig.

Forschungsfrage/Ziel: Braucht man zwingend autonome Agenten, um reale SE-Probleme zu lösen?

Methode: (1) **Lokalisierung** hierarchisch (Datei, dann Klasse/Funktion, dann Zeile). (2) **Reparatur**: mehrere Patch-Kandidaten im Search/Replace-Format. (3) **Validierung**: generierte Reproduktionstests, Regressionstests, Mehrheitsentscheid.

Daten: SWE-bench Lite (300), bereinigtes Lite-S (249), SWE-bench Verified (500).

Ergebnisse: Lite: **32,00 %** gelöst bei ca. **0,70 $** pro Aufgabe. Verified: 38,80 %. Dateilokalisierung 81,67 % genau.

Limitationen: Mögliche Datenleckage (Patches in Trainingsdaten), Evaluation fast nur auf SWE-bench, Schwächen bei Issues ohne Ortshinweise.

Begriffe: *Agentless vs. agentic*, *hierarchische Fehlerlokalisierung*.

Rolle in unserem Paper: Pol **geringer Autonomie** im Autonomiespektrum. Zeigt: Mehr Autonomie ist nicht automatisch besser.

Takerngsaksiri et al.: HULA

ICSE-SEIP 2025 (PDF: Preprint 2024) · Industrie

Worum geht's: Das Framework **HULA** integriert LLM-Agenten bei **Atlassian in JIRA**. Entwickler steuern Pläne und Code iterativ. Spart bei einfachen Aufgaben Zeit, die Codequalität erfordert aber weiter menschliche Kontrolle.

Forschungsfrage/Ziel: Wie effektiv, akzeptiert und nützlich sind Human-in-the-loop-Agenten im Unternehmen? (RQ1 offline ohne Feedback, RQ2 online mit Feedback, RQ3 Wahrnehmung per Umfrage.)

Methode: Drei Stufen: Offline-Evaluation, schrittweiser Online-Rollout in JIRA mit Interaktionsdaten, Entwicklerbefragung. Rollen: AI Planner Agent, AI Coding Agent, Human Agent.

Daten: Offline: SWE-bench Verified (500) und 369 interne JIRA-Issues (94 Repos, über 10 Sprachen). Online: 663 Issues, Rollout für bis zu 2.600 Entwickler. Umfrage: n = 109.

Ergebnisse: **82 % der Pläne** freigegeben (433/527), aber nur 25 % bis zum PR (95/376), davon **59 % gemergt** (56/95). 67 % der Befragten: menschliche Nacharbeit nötig.

Limitationen: Abhängig von detaillierten Issue-Texten (agile JIRA-Issues oft zu kurz). Schwächer bei mehrsprachigen Projekten. Unit-Tests als Maß unzureichend. Nur GPT-4 und Atlassian.

Begriffe: *Plan Approval Rate*, *File Localization*.

Rolle in unserem Paper: Stützt **P1** (gemeinsame Planung ist praktisch umsetzbar). Merke den Trichter: Plan freigegeben $\neq$ PR $\neq$ Merge.

Wang et al.: OpenHands (ehem. OpenDevin) ICLR 2025 · Plattform

Worum geht's: Quelloffene Plattform (MIT-Lizenz) für KI-Agenten, die wie Entwickler über Code, Terminal und Browser arbeiten, mit Docker-Sandbox, Multi-Agent-Delegation und Evaluationsframework.

Forschungsfrage/Ziel: Wie können Agenten sicher und effektiv Software erstellen und ändern? Ziel ist eine offene, erweiterbare Plattform.

Methode: Architektur aus Agenten-Abstraktion, *Event Stream* (Aktionen und Beobachtungen) und Runtime. Sandbox mit Bash, Jupyter und Browser. Standard-Agent **CodeActAgent**: Aktionen werden als ausführbarer Code formuliert.

Daten: 15 Benchmarks (u. a. SWE-bench Lite, HumanEvalFix, WebArena, GAIA).

Ergebnisse: SWE-bench Lite 26,0 % (mit Claude 3.5 Sonnet), HumanEvalFix 79,3 %.

Limitationen: Schwächen bei sehr langen Dateien, hoher manueller Aufwand für Workflows.

Begriffe: *Docker-Sandbox*, *CodeAct*, *Event Stream*.

Rolle in unserem Paper: Pol **hoher Autonomie**. Für **P2**: Die Sandbox schützt das System, macht den Code aber nicht sicher (Sajadi findet Lücken auch in OpenHands-Patches). Auch Agent in der Chen-Studie.

9.2 Produktivität und menschliche Aufsicht

Chen et al.: „Code with Me or for Me?" CHI '26 · kontrollierte Nutzerstudie

Worum geht's: Erster kontrollierter Nutzervergleich zwischen autonomem Agenten (OpenHands) und Copilot. Agenten halbieren die aktive Arbeit und lösen mehr Aufgaben, werfen aber Fragen zu Transparenz und Kontrolle auf. Die Rolle verschiebt sich vom Schreiben zum Überwachen.

Forschungsfragen: RQ1: Korrektheit und Nutzeraufwand? RQ2: kognitive Last, Arbeitsfluss, Zufriedenheit? RQ3: Wie ändern sich Interaktionsmuster?

Methode: **Within-Subject-Design**: Jede Person arbeitet je 40 Minuten mit Copilot und mit OpenHands (Claude 3.7 Sonnet), Reihenfolge randomisiert. Telemetrie, Video, Code-Snapshots, Likert-Fragebögen.

Daten: **20 Studierende** (über Uni-Mailinglisten rekrutiert), regelmäßige Copilot-Nutzer ohne Agentenerfahrung. 85 % hatten mindestens 3 Jahre professionelle Programmiererfahrung. 6 Aufgaben aus GAIA, SWE-bench und matplotlib.

Ergebnisse: Korrektheit **25 %** → **60 %** ($p = 0{,}02$). Aktive Zeit **25,1** → **12,5 min** ($p = 0{,}01$). Mit Agentenlaufzeit 27,9 min (laut unserem Paper). 75 % berichten geringere kognitive Last, aber 55 % verstanden Agenten-Ausgaben schlechter.

Limitationen: Je nur ein Werkzeug, Agenten-Neulinge, kurze synchrone Aufgaben.

Begriffe: *Within-Subject*, *aktiver Nutzeraufwand*, *Managerial Role*.

Rolle in unserem Paper: Befund 1: aktive Zeit ≠ Gesamtdauer. Achtung: Die „+35“ sind **Prozentpunkte**.

Cui et al.: Generative KI und hochqualifizierte Arbeit

Management Science 2026 (PDF: SSRN 2025) · 3 RCTs

Worum geht’s: Drei randomisierte Feldexperimente zu **GitHub Copilot** bei Microsoft, Accenture und einem anonymen Fortune-100-Unternehmen. Gepoolt +26,08 % erledigte Aufgaben. Weniger Erfahrene profitieren mehr.

Forschungsfrage/Ziel: Wie wirkt generative KI auf Produktivität (und Qualität) von Entwicklern im realen Arbeitsumfeld? Profitieren Weniger-Erfahrene stärker?

Methode: RCTs. Auswertung per gewichteter Instrumentalvariablen-Schätzung (2SLS) mit Fixed Effects, um unvollständige Compliance zu korrigieren.

Daten: **4.867** Entwickler (Microsoft 1.746, Accenture 320, anonym 3.054). GitHub-Aktivität: PRs, Commits, Builds.

Ergebnisse: **+26,08 % PRs** (SE 10,3 %), +13,55 % Commits, +38,38 % Builds. Juniors +21 bis 40 %, Seniors +7 bis 16 %.

Limitationen: Viel Rauschen (Wochen ohne Aktivität), imperfekte Compliance (Kontrollgruppe bekam teils früher Zugang), nur drei Firmen.

Begriffe: *LATE* (Local Average Treatment Effect: Effekt bei denen, die durch die Zuteilung tatsächlich nutzen), *Instrumentalvariable*.

Rolle in unserem Paper: Angrenzender Vergleich (Autocomplete, kein Agent). Kontrast zu Becker. PRs pro Woche sind ein **Output-Maß**, keine Zeitmessung.

Becker et al.: METR-Studie

arXiv 2025 · RCT

Worum geht’s: RCT zu KI-Werkzeugen von Anfang 2025 bei erfahrenen Open-Source-Entwicklern in ihren eigenen Projekten. Ergebnis: **19 % langsamer**, entgegen allen Erwartungen.

Forschungsfrage/Ziel: Wie wirken aktuelle KI-Tools tatsächlich auf die Produktivität erfahrener Entwickler in reifen Codebasen? Ziel: die Lücke zwischen Benchmarks und Praxis verkleinern.

Methode: Echte Issues werden **zufällig** auf „KI erlaubt“ oder „KI verboten“ verteilt. Log-lineare Regression der Bearbeitungszeiten, Analyse von 143 Stunden Bildschirmaufnahmen.

Daten: **16** Entwickler (im Schnitt 5 Jahre Erfahrung im Repo), **246** Aufgaben, große Repos (im Schnitt 23.000 Sterne, 1,1 Mio. Zeilen). Vor allem Cursor Pro mit Claude 3.5/3.7 Sonnet.

Ergebnisse: **+19 % Bearbeitungszeit**. Erwartet vorab: 24 % schneller. Rückblickend geschätzt: 20 % schneller. Experten prognostizierten 38–39 %. Unter 44 % der KI-Vorschläge akzeptiert, 9 % der Zeit für das Überarbeiten von KI-Code.

Limitationen: Nur erfahrene Entwickler in großen, vertrauten Projekten. Werkzeugstand Frühjahr 2025. Mögliche Artefakte des Versuchsaufbaus.

Begriffe: *RCT*, *Perceived Speedup*, *Tacit Knowledge* (implizites Projektwissen, das der KI fehlt).

Rolle in unserem Paper: Gegenbefund (Folie 5). Zeigt: Selbsteinschätzung ist unzuverlässig, objektive Zeitmessung ist nötig.

Dhanorkar et al.: Aufsicht in der Praxis arXiv 2026 · qualitative Interviews

Worum geht's: Wie beaufsichtigen Entwickler Software-Agenten tatsächlich? Aufsicht ist nicht nur reaktiv, sondern auch präventiv und proaktiv.

Forschungsfragen: RQ1: Wie beaufsichtigen Entwickler Agenten? RQ2: Welche Herausforderungen gibt es, und wie gehen sie damit um?

Methode: Halbstrukturierte 1:1-Interviews (ca. 60 min), induktives Kodieren (open, axial, selective).

Daten: **17** erfahrene Entwickler (12 von Microsoft), 12 davon nutzen Agenten täglich (Claude Code, Cursor, Cline, Copilot).

Ergebnisse: **Vier Aufsichtsformen**: A-priori-Kontrolle (Vorgaben vorab), Co-Planning (gemeinsame Planung), Echtzeit-Monitoring (selten), Post-hoc-Review. **Heuristiken beim Review**: Plan als Stellvertreter der Ausführung, bestandene Tests als Vollständigkeitsgarantie, Überfliegen („Eyeballing"), Vertrauen bei unbekanntem Kontext (Automation Bias).

Limitationen: 12 von 17 aus einem Konzern, überwiegend kleine Aufgaben, gut ausgestattete Umgebungen ohne Kostendruck.

Begriffe: *Co-Planning*, *Post-hoc Review*, *Automation Bias*.

Rolle in unserem Paper: Arbeitsverlagerung (Folie 2), **P1** und **P2**. Wichtig: Die Heuristiken (Plan- und Testvertrauen) sind genau die Fehlschlüsse, vor denen unser Paper warnt.

9.3 Kontext und Steuerung

Chatlatanagulchai et al.: Agentic Coding Manifests PROFES 2025 (PDF: arXiv) · Dokumentenanalyse

Worum geht's: Analyse von `CLAUDE.md`-Dateien („Manifeste"), die Agenten Kontext, Rolle und Projektregeln geben. Es gibt kaum Dokumentation dazu, wie man sie schreibt.

Forschungsfragen: RQ1: Wie sind Manifeste strukturiert? RQ2: Welche Arten von Anweisungen enthalten sie?

Methode: Messung der Markdown-Überschriftenebenen. Inhaltsklassifikation: LLM schlägt 80 Labels vor, Experten konsolidieren auf 15 Kategorien, drei Reviewer ordnen

zu.

Daten: **253** `CLAUDE.md`-Dateien aus 242 Repos (Filter: mind. 20 Commits nach Einführung).

Ergebnisse: Flache Struktur (Median eine H1, fünf H2). Häufigste Inhalte: Build/Run (77,1 %), Implementierungsdetails (71,9 %), Architektur (64,8 %), Tests (60,5 %). Selten: Performance (12,7 %), Security (8,7 %).

Limitationen: Nur ein Format (Claude Code). **Keine Messung der Wirkung** auf Agentenleistung.

Begriffe: *Agentic Coding Manifest*.

Rolle in unserem Paper: Zeigt, **was** in Kontextdateien steht (Abschnitt Kontext, P3), nicht *ob* es hilft.

Mohsenimofidi et al.: Context Engineering in OSS

MSR 2026 · Repository-Mining

Worum geht's: Verbreitung, Inhalt und Entwicklung von KI-Kontextdateien in Open-Source-Projekten. Frühe Adoptionsphase, kein einheitlicher Standard.

Forschungsfragen: RQ1: Wie verbreitet? RQ2: Welche Inhalte und Schreibstile? RQ3: Wie entwickeln sich die Dateien über die Zeit?

Methode: Scan nach Copilot-Instructions, `CLAUDE.md`, `AGENTS.md`, `GEMINI.md`. Qualitative Kategorisierung der Überschriften und Commit-Diffs.

Daten: 10.000 populäre Repos, davon **466** mit Kontextdatei. Detail: 155 `AGENTS.md`-Dateien, 453 Commits.

Ergebnisse: **ca. 5 %** Adoption (Copilot 229, `CLAUDE.md` 183, `AGENTS.md` 119, `GEMINI.md` 29). Inhalte: Konventionen, Contribution-Guides, Architektur. Fünf Schreibstile (deskriptiv, präskriptiv, prohibitiv, erklärend, konditional). **50 % der Dateien wurden nie geändert.**

Limitationen: Vier Formate, englischsprachige populäre Repos. **Keine Leistungsmessung.**

Begriffe: *Context Engineering*.

Rolle in unserem Paper: Verbreitung (466/10.000) und P3. Unsere Folgerung: nur Verbreitung, keine Wirkung. Dass die Hälfte nie aktualisiert wird, passt zur Forderung „klar und **aktuell**".

Gloaguen et al.: Evaluating AGENTS.md

ICLR-Workshop 2026 · kontrolliertes Experiment

Worum geht's: Helfen Kontextdateien Coding-Agenten wirklich? Entgegen den Empfehlungen der Anbieter steigern sie die Erfolgsrate im Allgemeinen nicht, erhöhen aber die Kosten um über 20 %.

Forschungsfrage/Ziel: Wie wirken **entwicklergeschriebene** und **LLM-generierte** Kontextdateien auf Erfolgsrate, Kosten und Verhalten von Agenten?

Methode: Vier Agent/Modell-Kombinationen (u. a. Claude Code/Sonnet 4.5, Codex/GPT-5.2, Qwen Code) in drei Settings: **keine** Datei, **LLM-generiert**, **Entwickler-Datei**. Trace-Analyse der Tool-Aufrufe.

Daten: **CTXBench**: neuer Benchmark, 138 Aufgaben aus 12 Python-Repos mit echten Kontextdateien. Dazu SWE-bench Lite (300).

Ergebnisse: LLM-Dateien ändern die Erfolgsrate kaum ($-0{,}5\,\%$ bzw. $-2\,\%$, n. s.). Entwickler-Dateien sind ca. 7 % besser als LLM-Dateien ($p = 0{,}038$). **Kosten +20 bis 23 %**, **Schritte +2,45 bis +3,92**. Anweisungen werden befolgt (z. B. das genannte Tool `uv`), relevante Dateien werden aber nicht schneller gefunden.

Limitationen: Nur Python (hohe Vertrautheit der Modelle könnte Effekte dämpfen). Effizienz und Sicherheit nicht gemessen.

Begriffe: *CTXBench*, *Trace-Analyse*, *Inferenzkosten*.

Rolle in unserem Paper: Befund 2 und **P3**. Kernsatz: n. s. ist kein Gleichheitsnachweis. Agenten *befolgen* Anweisungen, daher nur Unverzichtbares aufnehmen.

9.4 Benchmarks, Verifikation und Sicherheit

Jimenez et al.: SWE-bench

ICLR 2024 · Benchmark

Worum geht's: Benchmark aus **realen GitHub-Issues**. Ein Modell muss die Codebasis so ändern, dass die zugehörigen Tests bestehen. Damals löste das beste Modell (Claude 2) nur **1,96 %**.

Forschungsfrage/Ziel: Können Sprachmodelle reale, repository-weite Probleme lösen? Ziel: ein realistischer, ausführungsbasierter Benchmark.

Methode: Pipeline: Repo-Auswahl, dann Filter (PR löst Issue und enthält Tests), dann Ausführungsfilter (mind. ein Test wechselt von *fail* zu *pass*). Bewertung durch Einspielen des Patches und Testausführung.

Daten: **2.294** Aufgaben aus **12 Python-Repos** (Django, SymPy u. a.), im Schnitt 438.000 Zeilen Code. Teilmenge **Lite**: 300 Aufgaben.

Ergebnisse: Claude 2: 1,96 % (BM25-Retrieval), 4,80 % mit „Oracle"-Dateien. Längerer Kontext senkt die Leistung.

Limitationen: Nur Python. **Testbestehen garantiert keine Qualität**, Lesbarkeit oder Effizienz (sagen die Autoren selbst!).

Begriffe: *Fail-to-Pass-Test*, *ausführungsbasierte Evaluation*, *Patch*.

Rolle in unserem Paper: Zentrales **Messinstrument** vieler Studien. Grund für die Python-Limitation unseres Papers.

Yang et al.: SWE-agent

NeurIPS 2024 · System + Ablation

Worum geht's: LLM-Agenten brauchen eine speziell gestaltete Schnittstelle, ein **Agent-Computer Interface (ACI)**, um Software effektiv zu bearbeiten.

Forschungsfrage/Ziel: Wie beeinflusst das Schnittstellendesign die Leistung von LLM-Agenten?

Methode: Abstraktionsschicht über der Shell mit LLM-freundlichen Befehlen: Datei-Viewer, kompakte Suche, Editor mit **Linter-Guardrail** (Edits mit Syntaxfehler werden abgewiesen). **Ablationsstudien** gegen Shell-only und RAG.

Daten: SWE-bench (2.294), Lite (300), HumanEvalFix (164). GPT-4 Turbo, Claude 3 Opus.

Ergebnisse: SWE-bench 12,47 %. Lite **18,00 %** (Shell-only 11 %). **Ohne Linter 15 %**. Median 12 Schritte, ca. 1,21 $ pro erfolgreichem Lauf.

Limitationen: Kein Web-Browsing, keine statische Analyse. 52 % der Fehlschläge sind falsche oder zu spezifische Lösungen.

Begriffe: *ACI*, *Guardrail*, *Ablation*.

Rolle in unserem Paper: Stärkster Einzelbefund (P4): Linter 15 → 18 %, isoliert per Ablation.

Zhang et al.: AutoCodeRover

ISSTA 2024 · System + Evaluation

Worum geht's: Kombiniert LLMs mit **strukturierter Code-Suche über den Syntaxbaum (AST)**, um Issues autonom zu lösen. Hohe Lösungsrate bei geringen Kosten.

Forschungsfragen: RQ1: Effektivität und Kosten vs. Baselines? RQ2: Nutzen von testbasierter Fehlerlokalisierung (SBFL)? RQ3: Fehlerursachen?

Methode: (1) Kontextabruf durch schrittweise AST-Suche auf Klassen- und Methodenebene, (2) Patch-Erzeugung mit Syntaxprüfung und Retry, optional SBFL.

Daten: SWE-bench Lite (300), SWE-bench (2.294). GPT-4.

Ergebnisse: Lite **19 % (pass@1)**, **26 % (pass@3)**, mit SBFL 22 %. Ca. 0,43 $ und 195 s pro Aufgabe. **65,4 %** der plausiblen Patches korrekt (51 von 78).

Limitationen: Unvollständige Issue-Beschreibungen führen zu „Overfitting" (falscher Fix an der richtigen Stelle).

Begriffe: *plausibler vs. korrekter Patch*, *SBFL* (Spectrum-Based Fault Localization), *pass@k*.

Rolle in unserem Paper: Befund 3: Testbestehen ≠ Korrektheit. Die 65,4 % gehören zum **@3**-Versuch, nicht zum @1-Wert von 19 %!

Aleithan et al.: SWE-Bench+

arXiv 2024 · Benchmark-Analyse

Worum geht's: Qualitätsanalyse von SWE-bench. Viele „gelöste" Aufgaben sind durch **Lösungsleckage** oder **schwache Tests** verzerrt.

Forschungsfrage/Ziel: Lösen LLMs Issues wirklich selbst, oder verzerren Leckagen und schwache Tests die Erfolgsraten? Ziel: bereinigter Benchmark SWE-Bench+.

Methode: Manuelle Prüfung von 251 erfolgreichen Patches (SWE-agent + GPT-4). Neuer Datensatz aus Issues **nach** dem Trainings-Cutoff.

Daten: SWE-bench (2.294). SWE-Bench+: 548 neue Issues (Nov 2023 bis Aug 2024, 11 Projekte).

Ergebnisse: **32,67 %** der erfolgreichen Patches mit Lösung im Issue-Text, **31,08 %** mit schwachen Tests. Bereinigt fällt SWE-agent + GPT-4 von 12,47 % auf 3,97 %, auf SWE-Bench+ auf 0,55 %.

Limitationen: Schwache Tests bleiben auch im neuen Benchmark, nur Python.

Begriffe: *Solution Leakage*, *Weak Tests*, *Data Leakage*.

Rolle in unserem Paper: Benchmark-Kritik: Hohe SWE-bench-Werte überschätzen die tatsächliche Leistung.

Wang, Pradel, Liu: „Solved Issues“ wirklich gelöst? ICSE 2026 · empirische Studie

Worum geht's: Sind „gelöste“ Patches auf SWE-bench Verified wirklich korrekt? Wegen schwacher Testsuites oft nicht. Das neue Werkzeug **PatchDiff** findet Verhaltensabweichungen per differenziellem Testen.

Forschungsfragen: RQ1: Effekt der vollständigen Entwicklertests? RQ2: Wie häufig sind Verhaltensabweichungen? RQ3: Ursachen? RQ4: Anteil tatsächlich falscher Patches?

Methode: Alle Repo-Tests erneut ausführen. PatchDiff (LLM-basiert) erzeugt Tests, die generierten Patch und Entwickler-Patch unterscheiden. Manuelle Analyse und Taxonomie.

Daten: SWE-bench Verified (500), Patches von CodeStory, LearnByInteract, OpenHands. 877 plausible Patches, 77 manuell geprüft.

Ergebnisse: **7,8 %** scheitern an der vollen Testsuite. **29,6 %** weichen im Verhalten ab (Baseline-Tool nur 5,9 %). Von den geprüften abweichenden Patches sind **28,6 %** nachweislich falsch (22/77). Erfolgsraten sind im Schnitt um **6,4 Prozentpunkte** überhöht.

Limitationen: Kleines LLM (GPT-4o-mini) für PatchDiff, subjektive manuelle Einstufung, 30 %-Stichprobe. Bei 66,2 % bleibt es wegen unpräziser Issues unklar.

Begriffe: *Plausible Patch*, *differenzielles Testen*, *PatchDiff*.

Rolle in unserem Paper: **Befund 3** und **P4**: vollständige Testsuite vor Integration. Abweichung $\neq$ Fehler.

Cemri et al.: Warum scheitern Multi-Agent-Systeme? (MAST) NeurIPS 2025 D&B · Fehleranalyse

Worum geht's: Multi-Agenten-LLM-Systeme bringen oft kaum Gewinne gegenüber Einzelagenten. Die Fehlertaxonomie **MAST** (14 Fehlermodi, 3 Kategorien) zeigt: Viele

Fehler kommen aus dem **Systemdesign**, nicht aus dem Modell.

Forschungsfrage/Ziel: Warum versagen Multi-Agenten-Systeme, und welche Fehlermuster gibt es?

Methode: Grounded Theory auf 150 Traces, Übereinstimmung der Annotatoren $\kappa = 0{,}88$. Skalierung per LLM-as-a-Judge. Fallstudien mit Design-Interventionen.

Daten: **MAST-Data**: 1.642 annotierte Traces aus 7 Frameworks (u. a. ChatDev, MetaGPT). Fallstudie: 32 Aufgaben aus ProgramDev-v0.

Ergebnisse: Fehlerraten 41–87 %. Kategorien: Systemdesign 44,2 %, Fehlabstimmung zwischen Agenten 32,3 %, **Aufgabenverifikation 23,5 %** (fehlende oder falsche Prüfung). Design-Änderungen bringen bis zu **+15,6 Prozentpunkte** (ChatDev 25,0 → 40,6 %).

Limitationen: Taxonomie nicht vollständig, geschlossene Systeme schwer analysierbar, LLM-Judge kann Ursachen verwechseln.

Begriffe: *MAST*, *Inter-Agent Misalignment*, *Task Verification Failure*.

Rolle in unserem Paper: P4 (Gesamtauftrag gesondert prüfen). Die +15,6 PP stammen aus **mehreren** Änderungen, daher kein isolierter Nachweis.

Sajadi et al.: Wie sicher sind KI-Patches? arXiv 2025 · Sicherheitsanalyse

Worum geht's: Erste großflächige Sicherheitsanalyse KI-generierter Patches auf SWE-bench. LLMs und Agenten führen **mehr und neue Sicherheitslücken** ein als Entwickler.

Forschungsfragen: RQ1/2: Wie sicher sind Patches eines Standalone-LLMs (Llama 3.3) und von Agenten im Vergleich zu Entwicklern? RQ3: Welche Code-, Issue- und Projektmerkmale begünstigen unsichere Patches?

Methode: Vorher/Nachher-Vergleich mit drei statischen Analysetools (CodeQL, Semgrep, Bandit), Mehrheitsentscheid und manuelle Validierung.

Daten: SWE-bench (ca. 21.000 Issues). Llama 3.3 70B, OpenHands, AutoCodeRover, HoneyComb.

Ergebnisse: Llama: **135** neue Lücken vs. 12 bei Entwicklern. Agenten (Testset): OpenHands 44, AutoCodeRover 3, HoneyComb 2. Häufigste: OS Command Injection (CWE-78), Eval Injection (CWE-95). Unsichere Llama-Patches ändern mehr Dateien (1,72 vs. 1,25).

Limitationen: Ungenauigkeit statischer Tools, Agenten nur auf dem Testset, Auswahl an Modellen.

Begriffe: *CWE*, *Automated Program Repair (APR)*, *statische Analyse*.

Rolle in unserem Paper: P2 und **P5**: Sicherheit als eigener Endpunkt. Dateizahl und Lücken sind nur **assoziiert**. Modelle unterscheiden sich, daher keine kausale Aussage über Autonomie.

9.5 Integration in Pull Requests

Watanabe et al.: Agentic Coding in GitHub-PRs TOSEM 2026 · PR-Analyse

Worum geht's: Nutzen und Akzeptanz von **Claude-Code-PRs** auf GitHub. 83,8 % werden gemergt, aber 45,1 % der gemergten brauchen Nachbesserung.

Forschungsfragen: RQ1: Unterschiede zu menschlichen PRs in Größe und Zweck? RQ2: Ablehnungsrate und -gründe? RQ3: Anteil ohne Revision gemergt? RQ4: Welche Änderungen sind bei Revisionen nötig?

Methode: Quantitativ (Mann-Whitney-U, Chi-Quadrat) und qualitativ (manuelle Klassifikation, Open Coding).

Daten: **567** Agenten-PRs aus 157 Repos und 567 gematchte menschliche PRs (gleiche Autoren/Repos).

Ergebnisse: Merge-Rate 83,8 % vs. 91,0 % (Mensch). **45,1 %** der gemergten Agenten-PRs revidiert, Revisionsarten: **Bugfixes 47,7 %**, Doku 29,0 %, Refactoring 27,1 %. Agenten-PRs haben öfter mehrere Zwecke (39,9 % vs. 12,2 %) und viel längere Beschreibungen. Ablehnungsgründe u. a. alternative Lösung, zu große PRs.

Limitationen: Nur Claude Code mit explizitem Marker, Survival Bias (offene PRs ausgeschlossen), keine Langzeitdaten.

Begriffe: *Agentic PR (APR)*, *Revision vor Merge*.

Rolle in unserem Paper: Befund 3 und **P5**. Mehrzweck-PRs und „zu groß" als Ablehnungsgrund stützen **kleine, sachlich abgegrenzte PRs**.

Li et al.: AIDev / „AI Teammates in SE 3.0" arXiv 2025 · PR-Datensatz

Worum geht's: Agenten werden zu „KI-Teammates". Der Datensatz **AIDev** (über 456.000 PRs von fünf Agenten) zeigt eine Lücke zwischen Benchmark-Erfolg und realer Akzeptanz.

Forschungsfrage/Ziel: Wie wirken autonome Agenten in der Praxis auf Produktivität, Review-Latenz, Akzeptanz und Komplexität im Vergleich zu Menschen?

Methode: GitHub-Mining nach Bot-Markern, LLM-Aufgabenklassifikation, statistische Tests (Mann-Whitney-U, Cliff's Delta).

Daten: 456.535 Agenten-PRs (Codex, Devin, Copilot, Cursor, Claude Code) aus 61.453 Repos. Detail-Teilmenge: Repos mit über 500 Sternen.

Ergebnisse: Akzeptanz: Mensch 76,8 %, Codex 64 %, Devin 49 %, Copilot 35 %. Akzeptierte Codex-PRs schließen im Median nach **0,3 h** (Mensch 3,9 h). Nur 9,1 % der Codex-PRs ändern die Komplexität (Mensch 23,3 %).

Limitationen: Lokale Nutzung (Cursor, Claude Code) unvollständig erfasst, uneinheitliche Attribution.

Begriffe: *SE 3.0*, *Ecological Validity Gap*.

Rolle in unserem Paper: Proxy-Kritik: Schließlatenz ≠ Arbeitszeit oder Durchsatz. Fehlende Review-Aktion (58,2 % / 75,3 %) ≠ kein Review.

Zusatz: Huang et al.: AgentCoder arXiv 2023 · **nicht im Paper zitiert**

Worum geht's: Multi-Agenten-Codegenerierung mit Programmer, Test Designer und Test Executor. Tests werden *unabhängig* vom Code erzeugt, um Bias zu vermeiden.

Ergebnisse: HumanEval pass@1 96,3 % (GPT-4), deutlich weniger Tokens als MetaGPT.

Achtung: Liegt im Notebook, gehört aber **nicht** zu unseren 21 Arbeiten (u. a. von 2023 und Funktionsebene statt Repository). Falls gefragt: als Hintergrund zur Idee „unabhängige Tests", die zu P4 passt.

10 Die fünf Gestaltungsprinzipien

Grundregel für alle Prinzipien

Die Vorteile sind, soweit kein kontrollierter Einzelbefund vorliegt, **zu prüfende Erwartungen**. Jede *Vorschlag*-Regel ist eine **eigene Gestaltungsempfehlung**, **keine belegte Wirkungsaussage**.
Merkwort: **P**lanen – **A**utonomie – **K**ontext – **V**erifizieren – **I**ntegrieren („**PAKVI**").

P1: Planung gemeinsam prüfen

- **Inhalt:** Aufgabe, Zielanforderungen und Plan vor größeren Änderungen abstimmen.
- **Begründung:** HULA, Interviewstudie Dhanorkar. Diese Beteiligung ist praktisch umsetzbar.
- **Erwartung:** Missverständnisse werden früher erkannt. Eine **isolierte Zeit- oder Qualitätswirkung ist nicht belegt**.
- **Wichtig:** Das anschließende Code-Review bleibt erforderlich (Plan ≠ korrekte Umsetzung).
- **Vorschlag:** Bei unklaren Anforderungen Ziel und **Akzeptanzkriterien** vor der Umsetzung abstimmen.

P2: Autonomie nach Risiko und Prüfbarkeit staffeln

- **Inhalt:** Delegationsumfang und Eingriffspunkte richten sich nach **möglichem Schaden**, **Verifizierbarkeit** und **Review-Fähigkeit**.
- **Begründung:** Sicherheitsbefunde (Sajadi) und Aufsichtsbefunde (Dhanorkar) **motivie-**

ren die Regel, **testen sie aber nicht kausal**.

- **Sandbox:** OpenHands begrenzt Aktionen durch Docker. **Isolation macht den erzeugten Code nicht sicher.**
- **Vorschlag:** Sicherheitskritische oder **schwer rückgängig zu machende** Änderungen erfordern menschliche Freigabe.
- **Praxisbeispiel (eigene Überlegung):** Tests schreiben oder Doku anpassen darf der Agent autonom. Auth, Zahlungslogik, Datenbankmigrationen, Infrastruktur und Löschoperationen gehen nur mit Freigabe.

P3: Kontext gezielt ergänzen

- **Inhalt:** Erwartetes Verhalten, Einschränkungen, Build- und Testbefehle sollen **klar und aktuell** verfügbar sein. Kontextdateien enthalten vor allem **nicht anderweitig dokumentierte** Anforderungen.
- **Begründung:** Bestandsanalysen (Mohsenimofidi) und Kontextversuche (Gloaguen).
- **Nicht belegt:** eine allgemeine Steigerung der Erfolgsrate durch kürzere Dateien.
- **Vorschlag:** Fehlende Build-, Test- und Projektanforderungen gezielt ergänzen.

P4: Mehrstufig und am Auftrag verifizieren

- **Inhalt: statische Prüfungen** (Linter), **aufgabenbezogene Tests** und vor der Integration die **vollständige Testsuite** kombinieren. Ergänzende Tests prüfen die Anforderung über bestandene Fälle hinaus. Die **Erfüllung des Gesamtauftrags** wird gesondert beurteilt.
- **Begründung:** Yang (Linter), Wang (übersehene Defekte), Cemri (Gesamtauftrag).
- **Belegt:** einzelne Prüfgewinne und übersehene Defekte. **Offen:** ob diese Kombination die menschliche Nacharbeit verringert.
- **Vorschlag:** Bestandene Tests durch zusätzliche, **anforderungsbezogene** Prüfungen ergänzen.

P5: Änderungen überprüfbar integrieren

- **Inhalt: kleine, sachlich abgegrenzte PRs** mit **dokumentierter Agentenbeteiligung**, menschliches Review, risikogerechte Sicherheitsprüfungen.
- **Begründung:** Revisionsbefunde (Watanabe) und Sicherheitsbefunde (Sajadi) begründen den Prüfbedarf.
- **Nicht bestimmt:** optimaler Änderungsumfang und Nettozeitgewinn.

- **Vorschlag:** Änderungen sachlich abgrenzen und vor dem Merge nachvollziehbar prüfen.

Evidenzstärke pro Prinzip (eigene Einordnung für Fragen)

P	Evidenzart	Bewertung
P1	Industrieerfahrung + qualitativ	Umsetzbarkeit gezeigt, Wirkung nicht gemessen. **Eher schwach.**
P2	Beobachtend (Sicherheit) + qualitativ	Motiviert, nicht kausal getestet. **Eher schwach.**
P3	Kontrolliertes Experiment + Bestand	Gut untersucht, aber Ergebnis eher „kein Gewinn, mehr Kosten“. Stützt die *Zurückhaltung*.
P4	Ablation + empirische Patch-Analysen	**Am stärksten**: Linter-Effekt isoliert, Defekte trotz Tests belegt.
P5	PR-Beobachtungen + Sicherheit	Prüfbedarf belegt, optimaler Umfang offen. **Mittel.**

11 Diskussion, Evidenzgrenzen und Ausblick

11.1 Was trägt die Evidenz?

- **Stärkste direkte Hinweise:** eng begrenzte Interventionen (Linter in SWE-agent).
- **Nutzerexperimente:** Weniger aktive Arbeit garantiert keine kürzere Gesamtdauer.
- **Repository-Analysen:** praxisnah, aber wegen unterschiedlicher Aufgaben und **Selektionsprozesse** (z. B. welche Projekte Agenten nutzen) **nicht kausal**.
- **Proxies ersetzen nichts:** Akzeptanz, Testbestehen und Änderungsumfang ersetzen **weder Wartbarkeit noch Sicherheits- oder Zeitmessungen**.

11.2 Limitationen (Paper und Literatur)

1. Narrative Auswahl begrenzt Vollständigkeit und Reproduzierbarkeit.
2. Preprints tragen wesentliche Befunde, ihr Status ist zu beachten.
3. Modelle, Agentenversionen und Budgets unterscheiden sich.
4. SWE-bench mit Python-Repos dominiert, daher begrenzte Übertragbarkeit auf andere Sprachen und Industrie.
5. 17 Interviews sind keine repräsentative Häufigkeitsschätzung.

11.3 Forschungslücken

- **Welche einzelnen Gestaltungsentscheidungen** wirken unter **vergleichbaren Bedingungen**?
- Wie entwickeln sich **Prüfaufwand, Wartbarkeit und Folgedefekte langfristig**?
- Lange Repository-Beobachtungen beantworten diese Wirkungsfragen **nicht automatisch**.

11.4 Zentraler Zielkonflikt

Zielkonflikt

Zusätzliche Verifikation vs. Zeit- und Kostenaufwand. Entscheidend ist, ob zusätzliche Prüfschritte **genügend Defekte aufdecken**, um diesen Aufwand zu rechtfertigen.

11.5 Vorgeschlagene nächste Studie (sicher erklären können!)

Element	Ausgestaltung
Unabhängige Variable	Menschliche **Planfreigabe ja / nein** (allgemein: Planfreigabe, Kontext, Verifikation *einzeln* variieren).
Konstant halten	**Gleicher Agent, gleiches Modell, gleiches Budget**, gleicher Aufgabenpool.
Abhängige Variablen	**Funktionale Defekte**, **aktive menschliche Arbeitszeit**, **Gesamtdauer**, **Review-Aufwand**. Zusätzlich laut Paper: **Inferenzkosten**, **Sicherheitsbefunde**.
Langfristig	**Nachgelagerte Wartungsaufgaben** am erzeugten Code (Wartbarkeit, Folgedefekte).
Design-Ideen (eigene Ergänzung)	Randomisierte Zuteilung, ggf. Crossover (jede Person beide Bedingungen, Reihenfolge gegenbalanciert). Professionelle Entwickler, reale Repositories, mehrere Programmiersprachen. Defekte über versteckte Tests plus manuelle Prüfung erheben.
Warum gerade Planfreigabe?	Sie ist P1, wird in der Praxis genutzt (HULA, Dhanorkar), ist aber **nicht isoliert gemessen**. Freigegebene Pläne gelten als Heuristik, ohne dass ihre Fehlerrate bekannt ist.

12 Erwartete Fachfragen mit Musterantworten

Antwortschema „K–B–G“: Kernaussage, Beleg, Grenze

1. Kernaussage in einem Satz, **2. Beleg** mit Zahl und Quelle, **3. Grenze/Einschränkung**. Beispiel: „Nein, Testbestehen reicht nicht. Bei Wang et al. weichen 29,6 % der testbestehenden Patches im Verhalten ab. Eine Abweichung ist allerdings noch kein Fehlerbeweis: Von 77 geprüften waren 22 inkorrekt.“
Dauer: **30–60 Sekunden pro Antwort**. Lieber knapp und präzise als ausschweifend.

12.1 Grundverständnis

F1: Fassen Sie Ihr Paper in wenigen Sätzen zusammen.

Siehe Elevator Pitch (Abschnitt 2): 21 Arbeiten (2024–2026), drei Dimensionen, fünf Prinzipien. Belegt sind nur Einzelmaßnahmen, ein allgemeiner Gewinn ist nicht ableitbar. Endpunkte müssen getrennt gemessen werden.

F2: Was unterscheidet einen Coding-Agenten von GitHub Copilot (Autocomplete)?

Ein Agent ist ein LLM-basiertes System, das **mehrschrittig** plant, **Werkzeuge ausführt** (Dateien ändern, Befehle, Tests) und **Rückmeldungen der Umgebung** für weitere Schritte nutzt (Liu et al.). Autocomplete schlägt nur Code im Editor vor, der Mensch steuert jeden Schritt. Deshalb behandeln wir Codevervollständigung (Cui et al.) nur als angrenzenden Vergleich.

F3: Warum ist das Thema überhaupt relevant?

Laut Robbes et al. nutzen konservativ 22,20 % von 128.018 untersuchten GitHub-Projekten Coding-Agenten. Gleichzeitig verlagert sich Arbeit zum Spezifizieren, Überwachen und Prüfen (Dhanorkar). Ob das Zeit spart und korrekten Code liefert, zeigen Benchmark-Erfolge und PR-Zahlen nicht. Genau diese Lücke adressieren wir. Einschränkung: Die Stichprobe repräsentiert nicht die gesamte Softwareentwicklung.

F4: Was meinen Sie mit „Evidenzgrenzen“?

Die Grenze dessen, was die vorhandenen Studien tatsächlich belegen. Beispiele: Eine Studie misst aktive Zeit, aber nicht Gesamtdauer. Eine misst Testbestehen, aber nicht Korrektheit. Eine misst Merge-Raten, aber nicht Prüfaufwand. Wir trennen deshalb, was **kontrolliert belegt**, was **beobachtet** und was nur **qualitativ beschrieben** ist.

F5: Was ist ein „Endpunkt“ und warum betonen Sie die Trennung so stark?

Ein Endpunkt ist die Zielgröße einer Studie, z. B. Erfolgsrate, aktive Zeit, Gesamtdauer, Kosten oder Sicherheitslücken. Werden sie vermischt, entstehen Fehlschlüsse. Bei Chen sinkt die aktive Zeit von 25,1 auf 12,5 min, die Gesamtdauer mit Agent liegt aber

bei 27,9 min. Bei Gloaguen bleibt die Erfolgsrate n. s., die Kosten steigen aber um 20–23 %. Ein einziger Endpunkt erzählt also nie die ganze Geschichte.

12.2 Methodik und Kritik

F6: Wie sind Sie bei der Literaturrecherche vorgegangen?

ACM Digital Library, IEEE Xplore und arXiv mit kombinierten Suchbegriffen (z. B. *coding agent* und *human-in-the-loop*, *productivity*, *code quality*), dazu Rückwärtssuche. Nach Duplikatentfernung haben wir erst Titel/Abstract und dann Volltexte geprüft. Einschlusskriterien: ab 2024, Englisch oder Deutsch, mehrschrittige werkzeugnutzende LLM-Agenten, empirisch oder als System mit Evaluation, Bezug zu Workflow, Qualität oder Produktivität. Ergebnis: 21 Arbeiten, Stand 16.10.2026.

F7: Warum haben Sie kein systematisches Review (z. B. nach PRISMA) gemacht?

Das Format Short Paper und der Zeitrahmen erlaubten eine gezielte narrative Analyse. Ziel war eine **vergleichende Einordnung** der Evidenz, keine Vollerhebung. Wir berichten die Trefferzahlen pro Schritt nicht und beanspruchen deshalb **ausdrücklich keine reproduzierbare systematische Vollerhebung**. Das steht so im Paper. Ein systematisches Review mit dokumentierten Trefferzahlen wäre eine sinnvolle Erweiterung.

F8: Warum haben Sie keine Meta-Analyse bzw. keine gemeinsame Effektgröße berechnet?

Die Studien sind zu heterogen. Sie messen unterschiedliche Endpunkte mit unterschiedlichen Werkzeugen, Modellen, Budgets und Populationen (z. B. PRs pro Woche bei Cui vs. Bearbeitungszeit bei Becker). Ein Mittelwert darüber würde Äpfel mit Birnen vergleichen. Ein Architekturvergleich isoliert z. B. keinen Autonomieeffekt, wenn gleichzeitig das Modell wechselt.

F9: Gefordert war Konferenzliteratur. Warum nutzen Sie Preprints und Journals?

Konferenzbeiträge bilden die **technische Grundlage** (ICSE, FSE, ISSTA, MSR, NeurIPS, ICLR, CHI). Journals und **ausdrücklich gekennzeichnete** Preprints ergänzen Praxis- und Nutzerbefunde. Das Feld entwickelt sich sehr schnell, und wichtige Befunde wie die METR-Studie oder die Sicherheitsanalyse liegen (noch) als Preprint vor. In der Diskussion weisen wir darauf hin, dass ihr Status bei der Gewichtung zu beachten ist. Alle Quellen erfüllen die Vorgabe „nicht älter als 2024“.

F10: Ist Ihre Auswahl nicht anfällig für Selektionsbias?

Ja, das ist eine echte Limitation narrativer Reviews, und wir nennen sie im Paper. Gegenmaßnahmen: explizite Einschlusskriterien, mehrere Datenbanken, Rückwärtssu-

che und die bewusste Aufnahme von **Gegenbefunden**. Becker (+19 % Zeit) steht z. B. neben Cui (+26 % Aufgaben). Zudem bilden wir aus der Auswahl keine Effektgrößen, sondern ordnen jede Studie einzeln mit ihren Grenzen ein.

F11: Wie haben Sie die Qualität bzw. Stärke der Evidenz bewertet?

Durch Einordnung nach **Evidenzart**: kontrollierte Vergleiche (Experimente, Ablationen, RCTs), deskriptive Beobachtungen (Repository-/PR-Analysen) und qualitative Aussagen (Interviews). Pro Studie haben wir Design, Werkzeug, Population/Benchmark, Endpunkt und Einschränkungen gegenübergestellt. Ein formales Bewertungsschema (z. B. Risk-of-Bias-Tool) haben wir nicht verwendet. Das wäre ein Verbesserungspunkt.

F12: Warum genau 21 Arbeiten? Ist das genug?

Die Zahl ergibt sich aus den Einschlusskriterien und dem Fokus auf die drei Dimensionen. Für eine vergleichende narrative Synthese im Short-Paper-Format reicht sie, um die zentralen Befundtypen abzudecken: Nutzerstudien, Feldexperimente, Benchmarks, Patch-, PR- und Kontextanalysen. Vollständigkeit beanspruchen wir nicht.

12.3 Fragen zu den Befunden

F13: Spart ein Coding-Agent laut Chen et al. nun Zeit oder nicht?

Das hängt vom Endpunkt ab. Die **aktive** menschliche Zeit sinkt bei erfolgreichen Aufgaben von 25,1 auf 12,5 min. Mit Agentenlaufzeit liegt OpenHands aber bei **27,9 min** und damit über den 25,1 min von Copilot. Die **Gesamtdauer** ist also nicht kürzer. Der Mensch kann die Wartezeit aber evtl. anders nutzen. Grenzen: 20 Studierende, unterschiedliche Erfolgsquoten (der Zeitvergleich bezieht sich nur auf erfolgreiche Aufgaben), begrenzte Übertragbarkeit auf Profis.

F14: Cui findet +26 %, Becker −19 %. Widersprechen sich die Studien?

Nicht unmittelbar. Sie unterscheiden sich in drei Punkten. **Werkzeug:** Copilot-Autocomplete vs. überwiegend Cursor (Anfang 2025). **Population:** 4.867 Entwickler in Unternehmen vs. 16 erfahrene Open-Source-Entwickler in ihren eigenen, vertrauten Projekten. **Zeitmaß:** wöchentliche PRs als Output-Maß vs. Bearbeitungszeit pro Aufgabe. Folgerung: Werkzeugtyp, Population und Zeitmaß müssen bei der Übertragung beachtet werden.

F15: Warum glaubten die Entwickler bei Becker, 20 % schneller zu sein, obwohl sie 19 % langsamer waren?

Das Paper erklärt die Ursache nicht. Es zeigt aber: **Subjektive Selbsteinschätzung ist kein verlässliches Produktivitätsmaß**, deshalb braucht es objektive Zeitmessung. Mögliche Erklärungen (vorsichtig formulieren): Weniger aktive Tipparbeit fühlt

sich schneller an, obwohl Warten, Prompten und Prüfen der KI-Ausgaben Zeit kosten. Bei sehr vertrauten, großen Projekten bringt die KI zudem wenig Zusatzwissen.

F16: Lohnen sich Kontextdateien wie `AGENTS.md` oder `CLAUDE.md`?

Laut Gloaguen gibt es gegenüber keiner Kontextdatei **keinen signifikanten Gesamteffekt** auf die Erfolgsrate. LLM-generierte Dateien erhöhen sogar Schritte (+2,45/+3,92) und Kosten (+20/+23 %). Von Entwicklern geschriebene Dateien sind auf CTXBench besser als LLM-generierte, aber nicht signifikant besser als keine. Fazit (P3): Kontextdateien **knapp halten** und nur Anforderungen aufnehmen, die der Agent **sonst nicht findet**, etwa Build-/Testbefehle oder Projektregeln. Ein optimaler Umfang ist nicht bestimmt.

F17: „Nicht signifikant“ heißt doch, Kontextdateien haben keinen Effekt?

Nein. Nicht signifikant bedeutet nur, dass der beobachtete Unterschied mit Zufall vereinbar ist. Es ist **kein Nachweis gleicher Leistung**. Ein kleiner Effekt könnte existieren, aber mit der Stichprobe nicht nachweisbar sein. Dasselbe gilt für Watanabe: Der nicht signifikante Unterschied im Revisionsumfang zu menschlichen PRs beweist keine Gleichheit des Aufwands.

F18: Warum ist Testbestehen kein Beweis für Korrektheit?

Tests sind ein Näherungsmaß und können Anforderungen unvollständig abdecken. Belege: Bei AutoCodeRover sind nur 51 von 78 testbestehenden Patches (65,4 %) manuell korrekt. Bei Wang et al. scheitern 7,8 % von 877 testbestehenden Patches an der vollständigen Testsuite, 29,6 % weichen im Verhalten ab. SWE-Bench+ zeigt zudem schwache Tests und Lösungen, die schon im Issue stehen. Daraus folgt P4: Tests um anforderungsbezogene Prüfungen ergänzen.

F19: Was ist differenzielles Testen und warum beweist eine Abweichung keinen Fehler?

Beim differenziellen Testen vergleicht man das Verhalten des generierten Patches mit der Referenzlösung der Entwickler auf denselben Eingaben. Eine Abweichung zeigt nur, dass sich der Patch **anders** verhält. Das kann auch eine alternative, korrekte Lösung sein. Bei Wang et al. waren von 77 manuell geprüften verdächtigen Patches 22 inkorrekt, 4 korrekt und **51 unklar**.

F20: Was bedeutet @1 bzw. @3 bei AutoCodeRover?

@1 ist der Erfolg in einem einzelnen Lauf. @3 aggregiert drei Versuche (gelöst, wenn mindestens einer klappt). Die 65,4 % (51/78) beziehen sich auf die testbestehenden Patches aus dem aggregierten @3-Versuch und dürfen nicht mit der Erfolgsrate eines einzelnen Laufs verwechselt werden.

F21: Warum nennen Sie den Linter in SWE-agent den stärksten Befund?

Weil es eine **Ablation** ist: Nur der Linter wird variiert, alles andere bleibt gleich. Der Effekt ist damit **isoliert**: 15 % → 18 % gelöste Aufgaben auf SWE-bench Lite (+3 PP, relativ +20 %). Bei Cemri (25,0 → 40,6 %) wurden dagegen mehrere Dinge gleichzeitig geändert. Der Effekt ist größer, lässt sich aber keiner Einzelmaßnahme zuschreiben. Einschränkung: ein Benchmark, Python.

F22: Cemri zeigt +15,6 PP. Warum ist das kein Beleg für eine abschließende Zielprüfung?

Die verbesserte ChatDev-Topologie enthielt **mehrere Änderungen gleichzeitig**. Man kann den Gewinn daher nicht einer einzelnen Komponente wie der Zielprüfung zuschreiben. Außerdem sind es nur 32 Aufgaben aus ProgramDev-v0. Für P4 nutzen wir Cemri deshalb nur als Begründung, die **Erfüllung des Gesamtauftrags gesondert zu prüfen**, nicht als Wirkungsnachweis.

F23: Macht eine Sandbox wie bei OpenHands den Agenten sicher?

Sie schützt das **System**, weil die Aktionen des Agenten in einem Docker-Container isoliert ablaufen. Der **erzeugte Code** wird dadurch aber nicht sicher. Sajadi et al. finden Sicherheitslücken in Agenten-Patches. Deshalb verlangt P2 bei sicherheitskritischen oder schwer umkehrbaren Änderungen eine menschliche Freigabe, und P5 verlangt risikogerechte Sicherheitsprüfungen.

F24: Sind kleinere Patches also sicherer?

Das ist nicht belegt. Bei Llama-Patches gehen mehr geänderte Dateien mit mehr Sicherheitsproblemen einher. Das ist eine **Assoziation**, keine Kausalität, und eine Schutzwirkung kleinerer Patches wurde nicht gemessen. Auch über den Autonomiegrad lässt sich nichts Kausales sagen, weil Modelle und Mechanismen variieren. Kleine PRs (P5) empfehlen wir wegen der **besseren Prüfbarkeit**, nicht wegen eines nachgewiesenen Sicherheitseffekts.

F25: 45,1 % der Claude-Code-PRs wurden revidiert. Ist das schlecht?

Es zeigt vor allem, dass auch gemergte Agenten-PRs **Nacharbeit** brauchen. Häufigste Kategorie sind Bugfixes mit 47,7 %. Im Revisionsumfang ist der Unterschied zu menschlichen PRs nicht signifikant, aber das beweist keine Gleichheit. Ob 45,1 % „schlecht“ ist, kann man ohne vergleichbaren Aufwandsmaßstab nicht sagen. Wichtig ist: Merge $\neq$ fehlerfrei, daher P5.

F26: Li et al. zeigen schnellere Schließzeiten für Codex-PRs. Sind Agenten also produktiver?

Nein. Die Schließlatenz misst nur die Zeit vom Öffnen bis zum Schließen eines PRs, **weder menschliche Arbeitszeit noch Durchsatz**. Dass bei 58,2 % der Agenten-PRs

(und 75,3 % der menschlichen) keine explizite Reviewaktion sichtbar ist, heißt auch nicht, dass nicht geprüft wurde. Private Prüfung und Prüftiefe sind aus GitHub-Daten nicht ablesbar.

F27: Wie zuverlässig ist SWE-bench als Benchmark?

Er ist praxisnah, weil er reale GitHub-Issues verwendet, hat aber Grenzen. (1) Testbestehen ist nur ein Proxy, wie AutoCodeRover und Wang zeigen. (2) SWE-Bench+ belegt Lösungen in Issue-Texten und schwache Tests. (3) Er besteht aus **Python**-Repositories und ist damit begrenzt übertragbar. [Hintergrund, nicht im Paper]: Möglich ist auch Data Contamination, weil die Repositories öffentlich sind und in Trainingsdaten stecken können.

12.4 Fragen zu den Prinzipien

F28: Nennen Sie die fünf Prinzipien.

P1 Planung gemeinsam prüfen. P2 Autonomie nach Risiko und Prüfbarkeit staffeln. P3 Kontext gezielt ergänzen. P4 Mehrstufig und am Auftrag verifizieren. P5 Änderungen überprüfbar integrieren. (Merkwort **PAKVI**.)

F29: Wie sind Sie von den Befunden zu den Prinzipien gekommen?

Wir haben die Befunde entlang der drei Dimensionen Autonomie, Kontext und Qualitätssicherung gebündelt und aus jedem Problembereich eine Gestaltungsregel abgeleitet. Aufsicht/Planung (HULA, Dhanorkar) führt zu P1. Sicherheit/Aufsicht (Sajadi, Dhanorkar, OpenHands) führt zu P2. Kontextstudien (Gloaguen, Mohsenimofidi) führen zu P3. Testbefunde (Yang, Wang, Cemri) führen zu P4. PR- und Sicherheitsbefunde (Watanabe, Sajadi) führen zu P5. Jede Regel ist als *Vorschlag* markiert, weil ihre Wirkung nicht getestet ist.

F30: Wenn die Prinzipien nicht belegt sind: Warum sollte man ihnen folgen?

Sie sind die **bestbegründeten Empfehlungen auf Basis der aktuellen Evidenz**. Jedes Prinzip adressiert ein empirisch dokumentiertes Problem, etwa übersehene Defekte, Sicherheitslücken, Nacharbeit oder Kontextkosten. Belegt ist das **Problem**, nicht die Wirksamkeit der **Lösung als Gesamtworkflow**. Wir kennzeichnen das transparent und schlagen genau deshalb kontrollierte Studien vor.

F31: Welches Prinzip ist am besten, welches am schlechtesten belegt?

Am besten: P4. Die Linter-Ablation zeigt einen isolierten Effekt, und Wang/AutoCodeRover belegen übersehene Defekte trotz Tests. **Am schwächsten: P1 und P2**. Für P1 gibt es Industrieerfahrung und qualitative Interviews ohne Wirkungsmessung. P2 wird durch Sicherheits- und Aufsichtsbefunde motiviert, aber

nicht kausal getestet. P3 ist zwar experimentell untersucht, das Ergebnis lautet aber eher „kein Gewinn, mehr Kosten“.

F32: Stehen die Prinzipien nicht im Widerspruch zur Produktivität?

Genau das ist der zentrale **Zielkonflikt**: Mehr Verifikation, Planung und Review kosten Zeit und Geld. Entscheidend ist, ob zusätzliche Prüfschritte **genug Defekte aufdecken**, um den Aufwand zu rechtfertigen. Deshalb fordert P2 eine **risikogerechte** Staffelung: viel Prüfung bei kritischen, wenig bei harmlosen Änderungen.

F33: Wie würden Sie P2 in einem Unternehmen konkret umsetzen?

Eigene Ausgestaltung, nicht im Paper getestet: Änderungen nach Schadenspotenzial, Umkehrbarkeit und Prüfbarkeit einstufen. **Niedrig** (Tests, Doku, Refactoring mit guter Testabdeckung): Agent arbeitet autonom, normales Review. **Hoch** (Authentifizierung, Zahlungen, Datenmigrationen, Infrastruktur, Löschen): menschliche Planfreigabe vorab plus Sicherheitsprüfung vor dem Merge. Technisch zusätzlich Sandbox und eingeschränkte Berechtigungen. Die Sandbox ersetzt aber nicht die Prüfung des Codes.

F34: Was gehört in eine gute Kontextdatei?

Laut P3 vor allem, was der Agent **sonst nicht findet**: Build- und Testbefehle, verbindliche Projektkonventionen, Einschränkungen und erwartetes Verhalten. Das deckt sich mit den Inhalten der 253 analysierten `CLAUDE.md`-Dateien (Konventionen, Architektur, Build). Nicht hinein gehören Inhalte, die ohnehin im Code oder in der Doku stehen. Automatisch per LLM generierte Dateien haben bei Gloaguen Kosten und Schritte erhöht.

F35: Wie klein ist ein „kleiner“ PR?

Das ist offen, der optimale Änderungsumfang wurde in der Literatur nicht bestimmt. Wir empfehlen **sachlich abgegrenzte** PRs, also eine logische Änderung pro PR, weil sie besser prüfbar sind. Eine feste Zeilengrenze leiten wir bewusst nicht ab.

F36: Warum soll die Agentenbeteiligung im PR dokumentiert werden?

Damit Reviewer wissen, dass sie Code prüfen, der trotz bestandener Tests Defekte oder Sicherheitslücken enthalten kann (Wang, Sajadi), und ihre Prüftiefe entsprechend anpassen. Außerdem macht es spätere Auswertungen möglich, etwa wie sich Agenten-Code langfristig in der Wartung verhält.

12.5 Ausblick, Transfer und Meinung

F37: Welche Studie würden Sie als Nächstes durchführen?

Einen **kontrollierten Vergleich mit und ohne menschliche Planfreigabe** bei **gleichem Agenten, Modell und Budget**. Gemessen werden getrennt: funktionale Defekte, aktive menschliche Arbeitszeit, Gesamtdauer und Review-Aufwand, idealerweise auch Inferenzkosten, Sicherheitsbefunde und spätere Wartungsaufgaben. Design-Idee: randomisiert, professionelle Entwickler, reale Repositories in mehreren Sprachen.

F38: Welche Forschungslücken sehen Sie?

(1) Wirkung **einzelner Gestaltungsentscheidungen** unter vergleichbaren Bedingungen. (2) **Langfristige** Effekte auf Wartbarkeit, Prüfaufwand und Folgedefekte. (3) **Übertragbarkeit** über Python und SWE-bench hinaus auf andere Sprachen und Industrieaufgaben. (4) Wirkung der Prinzipien **als Gesamtworkflow**.

F39: Ihre Studien sind teilweise von Anfang 2025. Sind die Ergebnisse nicht schon veraltet?

Die konkreten Zahlen sind an Werkzeugstände gebunden, das sagen wir bei Becker ausdrücklich. Die **methodischen Kernaussagen** bleiben aber unabhängig vom Modell gültig: aktive Zeit $\neq$ Gesamtdauer, Testbestehen $\neq$ Korrektheit, Merge $\neq$ fehlerfrei. Gerade weil sich Modelle schnell ändern, braucht es Studien, die Modell und Budget konstant halten und einzelne Gestaltungsentscheidungen variieren.

F40: Werden Coding-Agenten Entwickler ersetzen?

Darüber trifft unser Paper keine Aussage. Was die Literatur zeigt: Die Arbeit **verlagert sich** zum Spezifizieren, Überwachen und Prüfen (Dhanorkar). Agenten-Code braucht weiterhin Review und Nacharbeit (45,1 % revidiert, Defekte trotz Tests). Menschliche Kompetenz in Anforderungen, Review und Sicherheit wird eher wichtiger.

F41: Würden Sie einem Unternehmen den Einsatz von Coding-Agenten empfehlen?

Ja, aber **gestaltet und gemessen**: nach P1–P5 und mit getrennter Messung von aktiver Zeit, Gesamtdauer, Defekten und Review-Aufwand. Einen pauschalen Produktivitätsgewinn sollte man nicht erwarten. Kontrollierte Befunde gibt es nur für einzelne Maßnahmen, und der Effekt hängt von Werkzeug, Population und Aufgabe ab (Cui vs. Becker).

F42: Was würden Sie an Ihrem eigenen Paper verbessern?

(1) Trefferzahlen pro Suchschritt berichten (PRISMA-artig), für mehr Reproduzierbarkeit. (2) Ein formales Schema zur Bewertung der Studienqualität. (3) Mehr Studien

außerhalb von Python/SWE-bench. (4) Eine Übersichtstabelle auch für die Qualitätsbefunde, nicht nur für Produktivität.

F43: Was ist Ihre persönliche Meinung bzw. Erfahrung mit Coding-Agenten?

Ehrlich und kurz antworten, vorher im Team absprechen. Gute Struktur: eigene Beobachtung → Bezug zum Paper. Beispiel: „Ich merke selbst, dass ich weniger tippe, aber mehr prüfe. Das passt zu Chen: Weniger aktive Arbeit heißt nicht automatisch weniger Gesamtzeit."

12.6 Fangfragen (ruhig bleiben, präzise korrigieren)

F44: „Also bringen Coding-Agenten nichts?"

Doch, so sagen wir das nicht. Unter konkreten Versuchsbedingungen zeigen sich **einzelne Verbesserungen**, z. B. 60 statt 25 % korrekt abgeschlossene Aufgaben bei Chen oder der Linter-Effekt. Wir sagen nur, dass sich daraus **kein allgemeiner** Produktivitäts- oder Qualitätsgewinn ableiten lässt.

F45: „Also machen KI-Werkzeuge Entwickler langsamer?"

Das gilt für die Becker-Studie: 16 erfahrene Entwickler, eigene vertraute Projekte, Werkzeuge von Anfang 2025. Cui findet in einer anderen Population mit anderem Werkzeug und Maß +26 %. Man kann keines der Ergebnisse verallgemeinern.

F46: „60 statt 25 %, der Agent ist also 2,4-mal besser?"

Relativ ja, +140 % bzw. +35 Prozentpunkte. Aber: 20 Studierende, bestimmte Aufgaben, und die Gesamtzeit ist mit 27,9 min nicht kürzer. „Besser" hängt vom Endpunkt ab.

F47: „Wenn der Plan freigegeben wurde, ist die Umsetzung doch korrekt?"

Nein. Freigegebene Pläne belegen keine korrekte Umsetzung. Dhanorkar beschreibt Plan-Vertrauen als Heuristik, misst aber keine Fehlerrate. Deshalb bleibt bei P1 das anschließende Code-Review erforderlich.

F48: „Ihre Prinzipien sind doch trivial, das macht man sowieso."

Teilweise sind sie gängige Praxis, das spricht für ihre Umsetzbarkeit. Unser Beitrag ist aber, sie **an konkrete Befunde zu binden** und offenzulegen, **was davon belegt ist und was nicht**. Gerade weil sie plausibel wirken, wird ihre Wirkung selten gemessen. Bei P3 zeigt Gloaguen sogar, dass eine plausible Praxis (mehr Kontext) nicht automatisch hilft.

13 Typische Denkfehler: niemals so sagen

Diese Gleichsetzungen sind FALSCH

1. Weniger aktive Arbeitszeit = kürzere Gesamtdauer (Chen: 12,5 vs. 27,9 min)
2. Tests bestanden = Code korrekt (65,4 %, 29,6 %, 7,8 %)
3. Verhaltensabweichung = Fehler (22 / 4 / 51 von 77)
4. Nicht signifikant = kein Effekt / gleich gut (Gloaguen, Watanabe)
5. Plan freigegeben = korrekte Umsetzung (Dhanorkar)
6. Sandbox = sicherer Code (OpenHands vs. Sajadi)
7. Mehr Dateien = mehr Lücken $\Rightarrow$ kleine Patches schützen (nur Assoziation)
8. PR gemergt = fehlerfrei (45,1 % revidiert)
9. Kürzere Schließlatenz = mehr Produktivität (Li)
10. Keine GitHub-Reviewaktion = kein Review (private Prüfung unsichtbar)
11. @3-Wert = @1-Erfolgsrate (AutoCodeRover)
12. Prozent = Prozentpunkte (25 $\rightarrow$ 60 % = +35 PP= +140 %)
13. Gefühlte Zeitersparnis = echte Zeitersparnis (Becker: +19 % vs. -20 %)
14. Autocomplete-Ergebnisse = Agenten-Ergebnisse (Cui)
15. 17 Interviews = Häufigkeitsaussage (qualitativ)
16. Unsere Prinzipien = bewiesener Workflow (*Vorschläge*!)
17. 22,2 % Adoption = Anteil der gesamten Softwareentwicklung (GitHub-Stichprobe)

Formulierungen, die immer passen

- „Unter diesen Versuchsbedingungen …“
- „Das ist eine Assoziation, keine kausale Aussage.“
- „Das ist ein Näherungsmaß für …, nicht … selbst.“
- „Das ist unser Vorschlag, die Wirkung ist nicht nachgewiesen.“
- „Das hat unser Korpus nicht untersucht. Messen könnte man es, indem …“

14 Strategie für die Fragerunde

- **Frage kurz wiederholen oder paraphrasieren.** Das verschafft Zeit und stellt sicher, dass du sie richtig verstanden hast.
- **K–B–G-Schema:** Kernaussage, Beleg (Zahl + Quelle), Grenze.
- **Nicht spekulieren als Fakt.** Wenn etwas nicht im Paper steht: „Das haben die Studien in unserem Korpus nicht untersucht. Ich würde vermuten …, prüfen könnte man das durch …“
- **Kritik annehmen statt verteidigen.** Bei Methodenkritik (narrativ, Preprints, Python) die Limitation bestätigen, auf die Stelle im Paper verweisen und eine Verbesserung nennen. Das wirkt souverän.
- **Kurz ergänzen ist ok, übernehmen nicht.** Jede Person muss ihre eigenen 1–2 Fragen selbst beantworten. Teammitglieder ergänzen höchstens einen Satz.
- **Zahlen nur nennen, wenn sicher.** Lieber „knapp zwei Drittel“ als eine falsche Kommazahl.
- **Folien als Stütze nutzen:** Bei Zahlenfragen auf Folie 5, 6, 7 oder 9 zurückspringen.

15 Schnelltest: Karteikarten

Die rechte Spalte abdecken und laut beantworten.

Frage	Antwort
Wie viele Arbeiten, welcher Zeitraum, welcher Stand?	21, 2024–2026, 16.10.2026
Welche drei Dimensionen?	Autonomie, Kontext, Qualitätssicherung
Welche drei Evidenzarten?	kontrolliert, deskriptiv-beobachtend, qualitativ
Welche Datenbanken?	ACM DL, IEEE Xplore, arXiv + Rückwärtssuche
Adoptionsrate laut Robbes?	22,20 % konservativ, 28,66 % obere Schätzung, 128.018 Projekte
Chen: Erfolgsraten?	Copilot 25 %, OpenHands 60 %
Chen: Zeiten?	aktiv 25,1 vs. 12,5 min, OpenHands gesamt 27,9 min
Becker: Ergebnis?	+19 % Zeit, geschätzt 20 % Ersparnis, 16 Devs, 246 Aufgaben

Frage	Antwort
Cui: Ergebnis?	+26,08 % Aufgaben (SE 10,3 %), 4.867 Devs, Autocomplete
Gloaguen: Kosten und Schritte?	+20/+23 % Kosten, +2,45/+3,92 Schritte (Lite/CTXBench)
Wie viele `CLAUDE.md` analysiert?	253
Kontextdateien-Verbreitung?	466 von 10.000 Projekten
AutoCodeRover korrekt?	51 von 78 = 65,4 % (@3)
Wang: Zahlen?	877 Patches, 7,8 % volle Suite, 260 = 29,6 % abweichend, 77 → 22/4/51
Linter-Effekt?	15 → 18 % auf SWE-bench Lite
Cemri?	32 Aufgaben, 25,0 → 40,6 %, mehrere Änderungen
Watanabe?	475/567 gemergt, 214 = 45,1 % revidiert, Bugfixes 47,7 %
Li?	kürzere Schließlatenz, 58,2 % (Agent) / 75,3 % (Mensch) ohne Reviewaktion
Wie viele Interviews bei Dhanorkar?	17
Vier Aufsichtsformen?	Vorgaben vorab, gemeinsame Planung, laufende Überwachung, nachträgliches Review
Agentless-Phasen?	Lokalisierung, Reparatur, Validierung
Was begrenzt OpenHands?	Aktionen per Docker-Sandbox, nicht die Code-Sicherheit
Zentraler Zielkonflikt?	Verifikation vs. Zeit-/Kostenaufwand
Nächste Studie?	mit/ohne Planfreigabe, gleicher Agent, Modell, Budget
Vier getrennte Messgrößen (Folie 10)?	Defekte, aktive Zeit, Gesamtdauer, Review-Aufwand
Was getrennt bewerten (Fazit)?	aktive Zeit, Gesamtdauer, Testbestehen, funktionale Korrektheit
Fünf Prinzipien?	Planung, Autonomie, Kontext, Verifikation, Integration (PAKVI)

16 Checkliste für den 12.11.

- ☐ Pitch (Abschnitt 2) frei in unter 45 Sekunden sprechen können.
- ☐ Eigenen Redeteil mit Stoppuhr: max. ca. 3:20 min.
- ☐ Übergabesätze zwischen den Sprechern abgesprochen.
- ☐ Alle Zahlen der Tabelle in Abschnitt 7 einmal laut wiederholt.
- ☐ Die fünf Prinzipien inkl. Vorschlag-Regel aufsagen können.
- ☐ Die drei wichtigsten Limitationen nennen können (narrativ ohne Trefferzahlen, Preprints, Python/SWE-bench).
- ☐ Nächste Studie (UV, konstant, AVs) erklären können.
- ☐ Fallen-Liste (Abschnitt 13) durchgelesen.
- ☐ Präsentation auf Laptop *und* USB-Stick, Paper als PDF griffbereit.

Viel Erfolg am 12.11.!